

Podstawy Programowania

Wykład nr 1: Wprowadzenie

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Podziękowania

Materiały do niniejszego wykładu zostały opracowane z wykorzystaniem fragmentów wykładów udostępnionych przez dr. hab. inż. Adriana Kosowskiego, dr. Krzysztofa Manuszewskiego i dr Olgi Choreń.

Zasady zaliczenia

Realizacja przedmiotu:

- wykład: 30 godzin (do połowy semestru)
- laboratorium zamknięte (LZ): 15 godzin (do połowy semestru)
- laboratorium otwarte (LO): praca indywidualna

Zaliczenie wykładu:

- w ramach zaliczenia wykładu odbędzie się kolokwium podstawowe (procentowy wynik punktowy oznaczmy przez W_1) oraz kolokwium poprawkowe (W_2)
- wynik z części wykładowej jest równy $W = W_1$ jeśli student nie pisał kolokwium poprawkowego oraz $W = W_2$ jeśli student pisał kolokwium poprawkowe
- wykład jest uznany za zaliczony jeśli $W \geq 50\%$

Zasady zaliczenia c.d.

Zaliczenie laboratorium zamkniętego:

- Laboratorium zamknięte jest uznane za zaliczone jeśli procentowy wynik, oznaczany przez LZ , wynosi co najmniej 50%.

Zaliczenie laboratorium otwartego:

- Laboratorium otwarte jest uznane za zaliczone jeśli procentowy wynik, oznaczany przez LO , wynosi co najmniej 50%.

Zaliczenie przedmiotu:

- Jeśli $W < 50\%$ lub $LO < 50\%$ lub $LZ < 50\%$, to ocena końcowa z przedmiotu wynosi 2.0.
- Jeśli $W \geq 50\%$, $LO \geq 50\%$ i $LZ \geq 50\%$, to obliczamy procentowy wynik z przedmiotu wg wzoru: $PP = 0.4 \cdot W + 0.3 \cdot LO + 0.3 \cdot LZ$ a następnie ocenę końcową z przedmiotu wyznaczamy następująco:
 - $PP \geq 90\% \Rightarrow$ ocena = 5.0
 - $80\% \leq PP < 90\% \Rightarrow$ ocena = 4.5
 - $70\% \leq PP < 80\% \Rightarrow$ ocena = 4.0
 - $60\% \leq PP < 70\% \Rightarrow$ ocena = 3.5
 - $50\% \leq PP < 60\% \Rightarrow$ ocena = 3.0
 - $PP < 50\% \Rightarrow$ ocena = 2.0

Cel przedmiotu i literatura

Cel przedmiotu:

- nauka podstaw programowania strukturalnego
- wprowadzenie do algorytmów i najprostszych struktur danych
- nauka języka C (z drobnymi 'zapożyczeniami' z języka C++)

Literatura:

- J. Grębosz: **Symfonia C++**. Programowanie w języku C++ orientowane obiektowo, wyd. Kallimach, Kraków
- B.W. Kernighan, D. M. Ritchie, **Język ANSI C**, Wydawnictwa Naukowo-Techniczne, Warszawa

Cykl wytwarzania oprogramowania

Na nasze potrzeby przytaczamy prosty model:

1. Projekt i analiza:
 - opis i sformułowanie problemu
 - opracowanie algorytmu
2. Implementacja.
 - programowanie
3. Testowanie i wdrożenie.
 - kompilacja
 - uruchamianie
 - testowanie

Algorytm — definicje

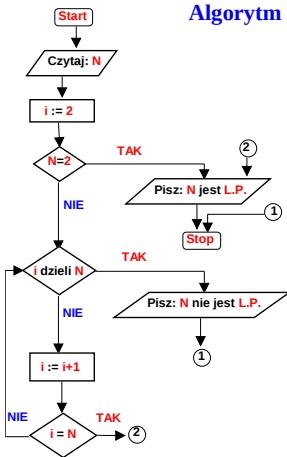
- Algorytm to skończony ciąg sekwencji/reguł, które aplikuje się do skończonej ilości danych, pozwalający rozwiązywać zbliżone do siebie klasy problemów.
- Algorytm to zespół reguł charakterystycznych dla pewnych obliczeń lub czynności informatycznych.

Algorytm:

- **przyjmuje dane wejściowe** (w ilości większej lub równej zero), pochodzące z dobrze zdefiniowanego zbioru
- **produkuje wynik**, niekoniecznie numeryczny,
- POWINIEN BYĆ **precyzyjnie zdefiniowany** – każdy krok algorytmu musi być jednoznacznie określony, deterministyczny
- POWINIEN BYĆ **skończony** – wynik algorytmu musi być 'kiedyś' dostarczony, algorytm ma własność stopu

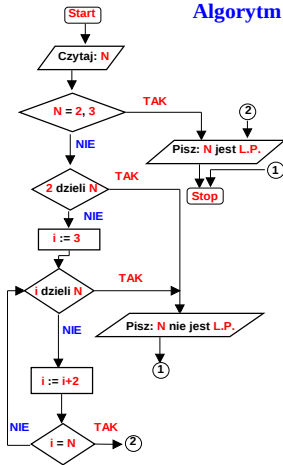
Różne sposoby rozwiązania problemu

Algorytm 1



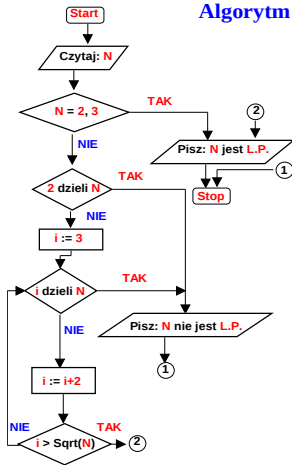
Różne sposoby rozwiązania problemu

Algorytm 2



Różne sposoby rozwiązania problemu

Algorytm 3



Różne sposoby rozwiązania problemu

- Algorytm 1 sprawdza wszystkie liczby i począwszy od 2 i skończywszy na $N - 1$
- Algorytm 2 sprawdza liczbę 2 oraz wszystkie liczby nieparzyste mniejsze niż N
- Algorytm 3 sprawdza liczbę 2 oraz wszystkie liczby nieparzyste mniejsze lub równe $\sqrt{N}$

N	Algorytm 1	Algorytm 2	Algorytm 3
10	8	5	2
100	98	50	5
1000	998	500	16

Uwaga: liczby iteracji podane w tabeli nie odzwierciedlają faktycznej liczby iteracji dla $N = 10, 100, 1000$, lecz stanowią poglądową ilustrację pesymistycznej liczby iteracji, która ma miejsce w sytuacji, gdy N jest liczbą pierwszą.

Zalecenia

- wybór algorytmu poprzedza kodowanie
- wybór algorytmu wpływa na: łatwość kodowania, szybkość działania i czytelność końcowego programu
- dbaj o uniwersalność programu (przenośność)
- dobry algorytm jest warunkiem **koniecznym** ale nie **dostatecznym** stworzenia dobrego programu

Język C

Cechy C:

- kompilacja do kodu maszynowego
- pre-procesor
- biblioteki standardowe (wejście wyjście, matematyczna, i inne)
- niewielka liczba słów kluczowych
- struktury, wskaźniki

Czego w C nie ma:

- wyjątków
- automatycznego odzyskiwania pamięci (tzw. garbage collector)
- elementów charakterystycznych dla programowania obiektowego

Język C

Rys historyczny

- 1972 – język C – Dennis Ritchie, Bell Labs (rozwinięty na potrzeby systemu Unix)
- 1978 – Brian Kernighan i Dennis Ritchie opracowują nieformalną specyfikację języka
- standaryzacja: 1989/1990 (ANSI/ISO), 1999 (“C99”), 2011 (“C11”)
- rozwinięcia: C++, Objective C
- języki wzorujące się: Java, C#, JavaScript, Perl, PHP, ...

Główne zastosowania

- programowanie systemowe (Unix/Windows), programowanie mikrokontrolerów, urządzeń osadzonych,...
- programowanie wydajnych aplikacji do przetwarzania danych, obliczeń numerycznych, aplikacji graficznych (C/C++)
- aplikacje ogólnego przeznaczenia

Podstawowe paradygmaty w języku C

Programowanie imperatywne: programista opisuje w jasny sposób ciąg instrukcji przetwarzania i sterujących (skoków, warunków, pętli, wywołań podprogramów) do wykonania:

- właściwy opis: $\text{silnia}(n)$: $\text{wynik} = 1$, w pętli dla i od 1 do n wykonuj: $\text{wynik} = i * \text{wynik}$, zwróć wynik
- niewłaściwy opis: $\text{silnia}(1) = 1$, $\text{silnia}(n) = \text{silnia}(n-1) * n$;

Programowanie strukturalne:

- program podzielony jest na funkcje/procedury, rozwiązujące konkretne podproblemy
- podział na funkcje, opcjonalnie grupowane w moduły, opisuje logiczną strukturę programu

Pierwszy program

```
#include <iostream>

int main() {
    std::cout << "Hello world!";
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello world!";
    return 0;
}
```

- oba powyższe programy są równoważne, tzn. efekt ich wykonania jest identyczny
- dyrektywa `#include` poleca dołączenie do programu odpowiedniej biblioteki (dołączenie pliku nagłówkowego związanego ze strumieniami C++)
- `'using namespace std'` wskazuje na przestrzeń nazw, w której znajduje się `cout`
- w niniejszym kursie będziemy korzystali z wejścia/wyjścia C++ (będzie to jedyny wyjątek)
- każdy program zawiera funkcję o nazwie `main`
- wykonanie programu zaczyna się od uruchomienia funkcji `main`
- instrukcje należące do funkcji ujęte są w nawiasy `{ }`
- do wypisania tekstu na ekran używamy strumieni C++

Styl programowania

- programy mają być czytane przez ludzi
- stosuj komentarze wstępne
- stosuj komentarze wyjaśniające
- komentarz - to nie parafraza instrukcji
- stosuj odstępy do poprawienia czytelności
- używaj dobrych nazw mnemonicznych
- wystarczy jedna instrukcja w wierszu
- stosuj wcięcia do uwidocznienia struktury programu
- PODPROGRAMY!!

Podstawy Programowania

Wykład nr 2: Pierwsze programy.

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Struktura programu

- Kod programu znajduje się w jednym bądź kilku plikach.
- Program składa się z dyrektyw preprocesora, definicji struktur, funkcji oraz zmiennych.
- Funkcje zbudowane są z instrukcji.
- Instrukcje składają się ze słów kluczowych, operatorów, nazw zmiennych oraz znaków grupujących i separujących (nawiasy, przecinki, średniki).
- Program zaczyna działanie od wykonania funkcji main.
- Pozostałe funkcje mogą być napisane przez programistę lub pochodzić z bibliotek.
- Przed uruchomieniem, program należy skompilować i scalić z funkcjami bibliotecznymi.

Drugi program

```
/* dyrektywy preprocesora: włączenie plików nagłówkowych */
#include <iostream>
#include <math.h>
using namespace std;

/* definicja funkcji main, zwracającej wartość całkowitą */
int main () {
    /* deklaracja zmiennych */
    double a, b;

    cout << "Podaj długości boków prostokąta\n";
    cin >> a;
    cin >> b;
    /* instrukcja przypisania, operacje arytmetyczne, wywołanie funkcji
       bibliotecznej */
    double c = sqrt(a*a + b*b);
    cout << "Długość przekątnej prostokąta wynosi: ";
    cout << c << '\n';
    return 0; /* wartość zwracana typu całkowitego */
}
```

Jednostki leksykalne

- Jednostki leksykalne to podstawowe “cegiełki”, z których zbudowany jest program.
- Jednostki leksykalne oddzielone są od siebie separatorami (spacja, przecinek, średnik, znak końca linii).
- Przykłady z poprzedniego programu:
 - `int`, `double`, `return` — słowa kluczowe
 - `main`, `sqrt` — identyfikatory (nazwy funkcji)
 - `a`, `b`, `c` — identyfikatory (nazwy zmiennych)
 - `0` — stały literał liczbowy
 - `"Podaj długości boków prostokąta--"` stały literał znakowy
 - komentarze są usuwane z programu przez preprocesor
 - dyrektywy są zastępowane odpowiednimi definicjami przez preprocesor

Identyfikatory

- Identyfikatory opisują nazwy zmiennych, funkcji, stałych
- Identyfikator jest sekwencją znaków alfanumerycznych, tj. liter i cyfr oraz znaków podkreślenia ('_')
- Pierwszy znak nie może być cyfrą
- Rozróżniane są małe oraz duże litery
- Długość identyfikatora jest nieograniczona, ale znaczące są tylko początkowe znaki identyfikatora (minimum 31 dla identyfikatorów zewnętrznych, minimum 63 w obrębie kodu)
- Należy unikać nazw zaczynających się od '_' (wykorzystywane w bibliotekach, zwłaszcza przy scalaniu z fragmentami kodu w innych językach)

Słowa kluczowe

- 32 słowa kluczowe w standardzie C89 (auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while)
- 5 kolejnych dodanych w standardzie C99 (_Bool, _Complex, _Imaginary, inline, restrict)
- 7 kolejnych w standardzie C11 (_Alignas, _Alignof, _Atomic, _Generic, _Noreturn, _Static_assert, _Thread_local)
- nie mogą być używane jako identyfikatory
- podstawowe typy danych: char, int, float, double, void
- złożone typy danych (użytkownika): struct, union
- aliasy typów danych: typedef, enum
- operator rozmiaru typu danych: sizeof
- modyfikatory typów numerycznych (domyślnie dla int): long, short, signed, unsigned
- modyfikatory zasięgu/widoczności zmiennych i funkcji: const, static, volatile, extern, register, auto
- strukturalne instrukcje sterujące: for, while, do, switch, case, default, if, else
- instrukcje sterujące skoków/powrotów: break, continue, return, goto

Deklaracja zmiennej

- Zmienne w języku C określają pewien obszar w pamięci komputera.
- Identyfikator zmiennej to nazwa, poprzez którą programista może odwoływać się do wartości zmiennej.
- W obrębie danego bloku instrukcji (pomiędzy klamrami { oraz } oraz wewnątrz niektórych instrukcji) identyfikator powinien być unikatowy.
- Wartości zmiennej mogą być modyfikowane w trakcie działania programu.
- Przykłady deklaracji zmiennych:

```
int i;  
char i,j;  
int i=0;  
char c = 'f';  
int i,j=0;
```

Instrukcja przypisania

```
zmienna = wyrażenie;
```

- $zmienna \Rightarrow$ l-wartość
- Jeśli typy zmiennej oraz wyrażenia są różne, to następuje automatyczna konwersja do typu zmiennej.
- różne warianty dla zmiennych liczbowych:

```
zmienna ◦= wyrażenie;
```

... gdzie ◦ zastępujemy operatorem, np.:

```
a *= 5;
```

- Przypisanie postaci $a \circ b$ jest interpretowane przez kompilator jako $a = a \circ b$.
W miejsce ◦ możemy wstawić: $+ - * / \% \ll \gg \& | ^$

Operatory arytmetyczne

- addytywne. Konieczna zgodność typów: oba operandy (a oraz b) typów arytmetycznych, lub jeden wskaźnikowy a drugi całkowity.
 - dodawanie: $a+b$
 - odejmowanie: $a-b$
- multiplikatywne
 - mnożenie: $a*b$ (operandy arytmetyczne)
 - dzielenie: a/b (operandy arytmetyczne) W przypadku operatorów całkowitych następuje zaokrąglenie w dół.
 - reszta z dzielenia liczby a przez liczbę b: $a\%b$ (operandy całkowite)
- inkrementacji/dekrementacji (zwiększenie/zmniejszenie o 1)
 - przedrostkowe: $--a$ lub $++a$. (Zmiana wartości a następuje **przed** obliczeniem wartości wyrażenia.)
 - przyrostkowe: $a--$ lub $a++$ (Zmiana wartości a następuje **po** obliczeniu wartości wyrażenia.)

Operatory arytmetyczne – przykład

```
#include <iostream>
using namespace std;

/* Program ilustrujący operatory arytmetyczne */
int main() {
    int a = 10, b = 7;

    b = (a++) % b;
    cout << "a=" << a << ",b=" << b << endl; /* a=11,b=3*/
    cout << "a=" << a << ",b=" << b++ << endl; /* a=11,b=3*/
    cout << "a=" << --a << ",b=" << b << endl; /* a=10,b=4*/
    a = 7/10;
    b = 7/2;
    cout << "a=" << a << ",b=" << b << endl; /* a=0,b=3*/

    return 0;
}
```

Instrukcje

- **Instrukcja** to wyrażenie zakończone średnikiem (który pomijamy w przypadku użycia grupowania).
- Instrukcje można grupować używając nawiasów klamrowych.
- Podstawowe typy instrukcji:
 - przypisania (=)
 - warunkowe (if, if-else)
 - wyboru (switch)
 - pętle (while, do-while, for)
 - skoki (goto, continue, break)

Operatory relacyjne

Operatory relacyjne:

- $a \leq b$ (a mniejsze lub równe b)
- $a \geq b$ (a większe lub równe b)
- $a < b$ (a mniejsze niż b)
- $a > b$ (a większe niż b)
- $a == b$ (a równe b)
- $a != b$ (a różne od b)

Oba operandy muszą być typów arytmetycznych lub muszą być wskaźnikami zgodnych typów.

Instrukcja warunkowa

```
if (wyrażenie)  
    instrukcja1;
```

lub

```
if (wyrażenie)  
    instrukcja1;  
else  
    instrukcja2;
```

1. Obliczana jest wartość wyrażenia w nawiasach.
2. Jeśli wartość ta jest **niezerowa**, to wykonywana jest instrukcja1.
3. Jeśli wartość ta jest równa zero, to:
 - W przypadku po lewej stronie: żadne dodatkowe instrukcje nie są wykonywane.
 - W przypadku po prawej stronie: wykonywana jest instrukcja2.

Instrukcja warunkowa – przykład

```
#include <iostream>
using namespace std;

/* Program ilustrujący użycie instrukcji warunkowej */
int main() {
    int liczba1, liczba2;

    cin >> liczba1;
    cin >> liczba2;

    if ( liczba2 != liczba1 )
        cout << "liczba1 jest różna od liczba2\n";
    if ( liczba2 == liczba1 )
        cout << "liczba1 == liczba2\n";
    return 0;
}
```

Instrukcja warunkowa – przykład

```
#include <iostream>
using namespace std;

/* Program ilustrujący użycie instrukcji warunkowej */
int main() {
    int liczba1 , liczba2;

    cin >> liczba1;
    cin >> liczba2;

    if ( liczba2 != liczba1 )
        cout << "liczba1 jest różna od liczba2\n";
    else
        cout << "liczba1 == liczba2\n";
    return 0;
}
```

Instrukcja warunkowa – przykład

```
#include <iostream>
using namespace std;

/* Program ilustrujący użycie instrukcji warunkowej */
int main() {
    int liczba1 , liczba2;

    cin >> liczba1;
    cin >> liczba2;

    if ( liczba2 - liczba1 )
        cout << "liczba1 jest różna od liczba2\n";
    else
        cout << "liczba1 == liczba2\n";
    return 0;
}
```

Zagnieżdżona instrukcja warunkowa

Uwaga! Porównaj:

```
if (wyrażenie1)
    instrukcja1;
else if (wyrażenie2)
    instrukcja2;
else if (wyrażenie3)
    instrukcja3;
else if (wyrażenie4)
    instrukcja4;
else
    instrukcja5;
```

```
if (wyrażenie1)
{
    if (wyrażenie2) instrukcja2;
}
else instrukcja5;
```

oraz

```
if (wyrażenie1)
    if (wyrażenie2) instrukcja2;
else instrukcja5;
```

Instrukcja for

```
for ( ini; wyrażenie; next )  
    instrukcja;
```

1. Relizacja instrukcji for rozpoczyna się od wykonania instrukcji **ini**.
 2. Obliczana jest wartość **wyrażenia**.
 3. Jeśli wartość ta jest **zerowa**, to następuje zakończenie wykonywania instrukcji for.
 4. Jeśli wartość ta jest **niezerowa**, to wykonywana jest **instrukcja** (treść pętli).
 5. Wykonywana (na zakończenie bieżącego obiegu pętli) jest instrukcja **next** i następuje powrót do pkt.2.
- Część **ini** lub **next** może się składać z kilku instrukcji – oddzielamy je przecinkami.
 - Każdą z części **ini**, **wyrażenie**, **next** można pominąć. Należy jednak pozostawić oddzielające średniki.
 - Jeśli **wyrażenie** jest puste, to jest ono traktowane jakby było prawdziwe.
 - Użycie instrukcji **break** wewnątrz pętli powoduje zakończenie wykonywania pętli.

Instrukcja for – przykład

```
#include <iostream>
using namespace std;

/* wypisanie liczb podzielnych przez 3 z zakresu [1,n],
   gdzie n jest wartością podaną przez użytkownika.*/
int main() {
    int n, i;

    cin >> n;
    for ( i=3; i <= n; i+=1 )
        if ( i % 3 == 0 )
            cout << i << endl;
    return 0;
}
```

Podstawy Programowania

Wykład nr 3: Operatory relacyjne, logiczne i bitowe. Instrukcje sterujące.

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Typy danych i kwalifikatory

Podstawowe typy danych w C:

- **char** – jeden bajt, typ znakowy
- **int** – typ całkowity, standard ANSI określa rozmiar na co najmniej dwa bajty (obecnie w większości kompilatorów cztery bajty)
- **float** – typ zmiennoprzecinkowy pojedynczej precyzji
- **double** – typ zmiennoprzecinkowy podwójnej precyzji

Kwalifikatory

- **short** (krótki, nie dłuższy niż int)
- **long** (długi, przynajmniej 4 bajty)
- **signed** (liczby ze znakiem)
- **unsigned** (liczby bez znaku)

Stałe

- Każda stała jest jakiegoś typu (np. występująca w wyrażeniach).
- Stałe całkowite, np.:
 - 1234 domyślnie typu `int`
 - 1234L typu `long int`
 - 1234U `unsigned int`
 - 1234UL `unsigned long int`
 - 012 system ósemkowy = 10
 - 0x12 system szesnastkowy = 18
- Stałe zmiennoprzecinkowe (domyślnie typu `double`), np.:
 - 314.15
 - 3.1415e2 (lub -3.1415E2) – notacja wykładnicza
 - 31415e-2
 - (użycie literki F lub L zmienia typ na `float` lub `long double`), np. 10.0F, 10.0L
- Stałe znakowe, np. 'a', 'b', '0',
- Stałe łańcuchowe, np. "ala ma psa"
- Znaki specjalne, np.: `\n` (nowa linia), `\r` (powrót karetki), `\t` (tabulacja), `\b` (backspace), `\?`, `\'`, `\"`, `\000` liczba ósemkowa, `\x000` liczba szesnastkowa

Kwalifikatory – przykłady

```
char c; /* zmienna z zakresu od -128 do 127 */
signed char c; /* równoważne powyższemu */
unsigned char c; /* zmienna z zakresu od 0 do 255 */

short int s; /* co najmniej 16 bitów, nie dłuższy niż int */
short s; /* równoważne powyższemu */
signed short s; /* równoważne powyższemu */

unsigned short us; /* jak short, lecz bez znaku */

int i;
signed int i; /* równoważne powyższemu */

long int l; /* co najmniej 32 bity, nie krótszy niż int */
long l; /* równoważne powyższemu */
signed long l;

long long int ll; /* co najmniej 64 bity */
long long ll; /* równoważne powyższemu */
signed long long ll; /* równoważne powyższemu */

unsigned long long ull; /* jak long long, lecz bez znaku */

float f;
double d;
long double ld;
```

Kwalifikator `const`

- Wartość zmiennej poprzedzonej kwalifikatorem `const` nie może być zmieniona w trakcie działania programu.
- Elementem dobrego stylu programowania jest używanie tego kwalifikatora w stosunku do takich zmiennych.
- Przykłady:

```
const double pi=3.14159;  
const int liczba_miesiecy=12;
```

Operator sizeof

Operator `sizeof` pozwala uzyskać rozmiar (w bajtach) danego typu. Pamiętaj, że rozmiary zmiennych zależą od konkretnej architektury. Można użyć poniższego programu, aby dowiedzieć się jakie są rozmiary poszczególnych typów na konkretnym komputerze.

```
#include <iostream>
using namespace std;
int main() { /* Uwaga: endl powoduje wypisanie znaku \n na ekran */
    cout << "Rozmiar typu char: " << sizeof(char) << endl;
    cout << "Rozmiar typu short: " << sizeof(short) << endl;
    cout << "Rozmiar typu int: " << sizeof(int) << endl;
    cout << "Rozmiar typu long: " << sizeof(long) << endl;
    cout << "Rozmiar typu long long: " << sizeof(long long) << endl;
    cout << "Rozmiar typu float: " << sizeof(float) << endl;
    cout << "Rozmiar typu double: " << sizeof(double) << endl;
    cout << "Rozmiar typu long double: " << sizeof(long double) << endl;
    return 0;
}
```

```
Rozmiar typu char: 1
Rozmiar typu short: 2
Rozmiar typu int: 4
Rozmiar typu long: 4
Rozmiar typu long long: 8
Rozmiar typu float: 4
Rozmiar typu double: 8
Rozmiar typu long double: 12
```

Przykładowe zakresy danych

W oparciu o powyższe rozmiary typów (na konkretnym komputerze, na którym uruchomiono powyższy program!) otrzymujemy następujące zakresy typów:

```
Rozmiar typu char: 1
Rozmiar typu short: 2
Rozmiar typu int: 4
Rozmiar typu long: 4
Rozmiar typu long long: 8
Rozmiar typu float: 4
Rozmiar typu double: 8
Rozmiar typu long double: 12
```

Typ	Zakres
char	od -128 do 127
unsigned char	od 0 do 255
short	od -32768 do 32767
unsigned short	od 0 do 65535
int	od -2147483648 do 2147483647
unsigned int	od 0 do 4294967295
long	od -2147483648 do 2147483647
float	$\pm 3.4 \cdot 10^{\pm 38}$
double	$\pm 1.7 \cdot 10^{\pm 308}$
long double	$\pm 1.2 \cdot 10^{\pm 4932}$

Przekroczenie zakresu typu

Uwaga: Należy mieć świadomość tego, że typy mają ograniczony zakres i dobierać typy szacując maksymalną wartość jaką powinny przechować w trakcie działania programu.

Przykład:

```
#include <iostream>
using namespace std;

/* Program ilustrujący przekroczenie zakresu typu */
int main() {
    short s;
    unsigned int ui = 0;

    s = 128*256;
    cout << s << endl;

    ui = ui - 1;
    cout << ui << endl;
    return 0;
}
```

```
-32768
4294967295
```

Operatory logiczne

Operatory logiczne:

- koniunkcja: `a && b`
- alternatywa: `a || b`
- negacja: `!a`

Wartość wyrażenia logicznego jest typu `int`.

Tabela wartości wyrażeń logicznych:

a	b	koniunkcja	alternatywa	negacja a
fałsz	fałsz	fałsz	fałsz	prawda
fałsz	prawda	fałsz	prawda	prawda
prawda	fałsz	fałsz	prawda	fałsz
prawda	prawda	prawda	prawda	fałsz

Operator warunkowy (`?:`). Wyrażenie `a ? b : c` realizowane jest następująco:

- Obliczana jest wartość `a`.
- Jeśli `a  $\neq$  0`, to obliczana jest wartość `b`, natomiast `c` nie jest obliczane.
- Jeśli `a = 0`, to obliczana jest wartość `c`, natomiast `b` nie jest obliczane.

```
#include <iostream>
using namespace std;

/* Program ilustrujący operatory logiczne */
int main() {
    int x = sizeof(int);
    int y = (x>4 ? 1 : 0);

    y == 0 ? (cout << "int ma 4 bajty\n") : (cout << "int ma > 4 bajty");
    return 0;
}
```

Instrukcja wyboru switch

```
switch (wyrażenie)
{
    case wartosc1 : instrukcja1;
        break;
    case wartosc2 : instrukcja2;
        break;
    case wartosc3 : instrukcja3;
        break;
    ...
    case wartoscN : instrukcjaN;
        break;
    default : instrukcjaD;
}
```

Instrukcja switch – wyjaśnienia

- Realizacja instrukcji rozpoczyna się od obliczenia wartości wyrażenia umieszczonego w nawiasach po słowie switch.
- Wartość wyrażenia musi być typu porządkowego (char, int).
- Jeśli jego wartość odpowiada którejś z wartości podanej w jednej z etykiet wartosc1,...,wartoscN, wówczas wykonywane są instrukcje począwszy od tej etykiety.
- Wykonywanie tych instrukcji kończy się po napotkaniu instrukcji break. Działanie instrukcji switch zostaje wówczas zakończone.
- Etykiety warunków muszą być wartościami liczbowymi stałymi (np. 9, 'a').
- Jeśli wartość wyrażenia nie zgadza się z żadną z wartości wartosc1,...,wartoscN, wówczas wykonywane są instrukcje umieszczone po etykiecie default.
- Etykieta default może być umieszczona w dowolnym miejscu instrukcji switch, nawet na samym jej początku.
- Etykietę default można pominąć. Wówczas, jeśli w zbiorze etykiet case nie ma żadnej etykiety równej wartości wyrażenia, instrukcja switch nie będzie wykonana.
- Uwaga! Instrukcje występujące po etykiecie case nie muszą kończyć się instrukcją break. Jeśli jej nie umieścimy, to będą wykonywane instrukcje umieszczone pod następną etykietą case.

Instrukcja switch – przykład

```
#include <iostream>
using namespace std;

/* Program ilustrujący użycie instrukcji switch */
int main() {
    int ocena;

    cin >> ocena;

    switch ( ocena ) {
        case 2:
            cout << "<50%\n" ;
            break;
        case 3:
            cout << "50%–70%\n" ;
            break;
        case 4:
            cout << "70%–90%\n" ;
            break;
        case 5:
            cout << "90%–100%\n" ;
            break;
    }
    return 0;
}
```

Instrukcja while

```
while (wyrażenie)  
    instrukcja;
```

1. Następuje bliczenie wartości wyrażenia w nawiasie.
2. Jeśli wartość ta jest **zerowa**, to następuje zakończenie wykonywania instrukcji while.
3. Jeśli wartość ta jest **niezerowa**, to następuje wykonanie instrukcji, po czym następuje powrót do pkt.1.

Instrukcja while – przykłady

```
#include <iostream>
using namespace std;

/* wypisanie liczb podzielnych przez 3 z zakresu [1,n],
   gdzie n jest wartością podaną przez użytkownika
   — wersja błędna!! */
int main() {
    int n, i=3;

    cin >> n;
    while ( i <= n ) {
        if ( i % 3 == 0 )
            cout << i;
    }
    return 0;
}
```

Uwaga: program działa błędnie: jeśli podamy na wejściu liczbę n większą lub równą 3, to program nigdy się nie zakończy i nie wypisze żadnego znaku na ekran.

Instrukcja while – przykłady

Przykład poprawnie działającego programu:

```
#include <iostream>
using namespace std;

/* wypisanie liczb podzielnych przez 3 z zakresu [1,n],
   gdzie n jest wartością podaną przez użytkownika.*/
int main() {
    int n, i=3;

    cin >> n;
    while ( i <= n ) {
        if ( i % 3 == 0 )
            cout << i << endl;
        i = i+1;
    }
    return 0;
}
```

Instrukcja do-while

```
do  
    instrukcja  
while (wyrażenie);
```

1. Wykonywana jest instrukcja.
2. Następnie obliczana jest wartość wyrażenia w nawiasie.
3. Jeśli jego wartość jest **zerowa**, to następuje zakończenie wykonywania instrukcji do-while. Jeśli jego wartość jest **niezerowa**, to następuje powrót do pkt.1.

Instrukcja do-while – przykład.

```
#include <iostream>
using namespace std;

/* wypisanie liczb podzielnych przez 3 z zakresu [1,n],
   gdzie n jest wartością podaną przez użytkownika.*/
int main() {
    int n, i=3;

    cin >> n;
    if ( n < 3 )
        return 0;
    do {
        if ( i % 3 == 0 )
            cout << i << endl;
        i = i+1;
    } while ( i <= n );
    return 0;
}
```

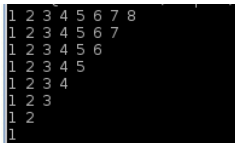
Instrukcja break

- Instrukcję **break** można używać w instrukcjach **for**, **while** oraz **do-while**.
- Jej wykonanie powoduje natychmiastowe zakończenie pętli, w której się znajduje.
- W przypadku pętli zagnieżdżonych, następuje zakończenie petli, w której została wykonania.

```
#include <iostream>
using namespace std;

/* Ilustracja działania instrukcji
   break w zagnieżdżonej pętli.*/
int main() {
    int i, j;

    for ( i=1; i <= 10; i+=1 ) {
        for ( j=1; j <= 10; j+=1 ) {
            if ( i + j >= 10 )
                break;
            cout << j << ' ';
        }
        cout << endl;
    }
    return 0;
}
```



```
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
1 2 3 4 5 6
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Instrukcja continue

- Instrukcję **continue** można używać w instrukcjach **for**, **while** oraz **do-while**.
- Jej wykonanie powoduje przerwanie bieżącej iteracji pętli, w której się znajduje.
- W przypadku pętli zagnieżdżonych, następuje przerwanie petli, w której została wykonania.
- W przeciwieństwie do instrukcji **break** nie następuje zakończenie wykonywania pętli.

Instrukcja 'zakazana': goto

```
goto etykieta;
```

- Instrukcja wykonuje skok do instrukcji opatrzonej **etykieta**.
- **etykieta** jest literałem, po którym następuje dwukropek.
- Etykieta musi znajdować się w "bieżącym zakresie ważności".
- Należy unikać używania tej instrukcji, gdyż utrudnia ona czytanie i analizę kodu.

Wskaźniki – bardzo krótkie wprowadzenie

- Dla każdego typu podstawowego (np. `int`) można zadeklarować wskaźnik na taki typ, czyli zadeklarować zmienną, która jest dedykowana do zapamiętywania adresu (w pamięci operacyjnej) zmiennej (np. typu `int`).
- Zmienną wskaźnikową typu `typ` deklarujemy: `typ *zmienna;`
- Przykład:
`int i;`
`int *wsk;`
- Dla dowolnej zmiennej, operator `&` zwraca adres tej zmiennej. Kontynuując przykład:
`wsk = &i;`
- Powyższe przypisanie powoduje, że zmienna `wsk` przechowuje adres zmiennej `i`.
- Operator `*` zastosowany do zmiennej wskaźnikowej powoduje odwołanie się do wartości **wskazywanej** przez tą zmienną (a nie do adresu przechowywanego we wskaźniku). Kontynuując przykład:
`*wsk = 1;`
- Powyższe przypisanie powoduje umieszczenie liczby 1 pod adresem przechowywanym w zmiennej `w`. Jako że zmienna `wsk` przechowuje adres zmiennej `i`, przypisanie to zmienia wartość zmiennej `i` (po tym przypisaniu `i` jest równe 1)

Styl programowania – przykład

```
Sum_Dod = 0; Licz_Dod = 0;
Sum_Poz = 0; Licz_Poz = 0;
for (i = 1; i <= N; i = i+1)
    if (a[i] > 0) {
        Sum_Dod = Sum_Dod + a[i];
        Licz_Dod = Licz_Dod + 1;
    } else {
        Sum_Poz = Sum_Poz + a[i];
        Licz_Poz = Licz_Poz + 1;
    }
```

```
Sum_Dod = 0; Licz_Dod = 0;
Sum_Poz = 0; Licz_Poz = 0;
i = 1;
et1: if (a[i] > 0) goto et2;
    Sum_Poz = Sum_Poz + a[i];
    Licz_Poz = Licz_Poz + 1;
    goto et3;
et2: Sum_Dod = Sum_Dod + a[i];
    Licz_Dod = Licz_Dod + 1;
et3: i = i + 1;
    if (i <= N) goto et1;
```

```
Sum_Dod = 0; Licz_Dod = 0; Sum_Poz
    = 0; Licz_Poz = 0; i = 1;
et1: if (a[i] > 0) goto et2;
    Sum_Poz = Sum_Poz + a[i];
    Licz_Poz = Licz_Poz + 1; goto et3;
et2: Sum_Dod = Sum_Dod + a[i];
    Licz_Dod = Licz_Dod + 1;
et3: i = i + 1; if (i <= N) goto
    et1;
```

Zalecenia:

- Stosujemy konsekwentnie wcięcia.
- Unikamy stosowania instrukcji `goto`.
- Nazwy zmiennych powinny sugerować ich przeznaczenie.
- Piszemy jedną instrukcję w jednej linii.

Efekty uboczne

Zasada: wartość zmiennej, która ulega modyfikacji w obrębie wyrażenia, może być pobrana (użyta) w tym wyrażeniu co najwyżej jeden raz! Dotyczy to przede wszystkim operatorów inkrementacji i dekrementacji.

Przykłady błędnych instrukcji:

- `a = (b++) + b;` Wartość zmiennej `b` nie jest jednoznaczna w kontekście operatora dodawania.
- `f() + g();` Nieokreślona kolejność wywołania funkcji `f` i `g`.
- `f(i,i=3);` Niejednoznaczna wartość pierwszego argumentu.

Podstawy Programowania

Wykład nr 4: Pozostałe operatory, arytmetyka, konwersje pomiędzy typami

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Operatory bitowe & ^ | ~

- & realizuje operację logicznej koniunkcji (AND)
- ^ realizuje operację sumy modulo 2 (EXOR)
- | realizuje operację logicznej alternatywy (OR)
- ~ realizuje operację logicznej negacji (NOT)

Definicja spójników logicznych:

a	b	AND	EXOR	OR	NOT a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	0	1	0

Przykłady (przyjmujemy tutaj, że typ int jest 32 bitowy):

```
a = 13 & 5;           /*a przyjmuje wartość 5*/  
a = 9 | 37;           /*a przyjmuje wartość 45*/  
a = 11 ^ 18;          /*a przyjmuje wartość 25*/  
a = 21 & (~4);        /*a przyjmuje wartość 17*/
```

Przesunięcia bitowe $\gg$ oraz $\ll$

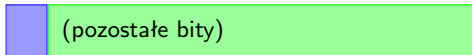
- Wyrażenie: $a \gg b$ (operandy całkowite) zwraca liczbę, której reprezentacja bitowa powstaje poprzez przesunięcie bitów liczby a o b pozycji w **prawo**.
- Wyrażenie: $a \ll b$ (operandy całkowite) zwraca liczbę, której reprezentacja bitowa powstaje poprzez przesunięcie bitów liczby a o b pozycji w **lewo**.

Uwagi:

- Operand b nie może być ujemny.
- Jeśli a jest typem ze znakiem, ma nieujemną wartość i $a \cdot 2^b$ jest reprezentowalne w wynikowym typie, to jest to wynikowa wartość operacji $a \ll b$. W innym przypadku rezultat jest niezdefiniowany dla ujemnych a .
- Jeśli a jest typem ze znakiem i ma nieujemną wartość, to wynik $a \gg b$ jest zależny od implementacji.
- Operacja $a \gg 1$ jest równoważna $a/2$.
- Operacja $a \ll 1$ jest równoważna $a*2$.
- Operatory przesunięć bitowych nie zmieniają wartości operandów (dotyczy sytuacji, gdy są to zmienne).

Reprezentacja liczb całkowitych

Bity numerujemy od zera (skrajnie prawy bit jest bitem zerowym). Bit znaku ma numer 31 dla 4-bajtowych typów (np. int na niektórych architekturach), 15 dla 2-bajтового typu (np. short int na niektórych architekturach) oraz numer 7 dla typu char.



bit znaku (=0 dla liczby nieujemnej oraz =1 dla liczby ujemnej)

```
#include <iostream>
using namespace std;
/* Program ilustrujący reprezentację binarną liczby typu int */
int main() {
    int a = -1, b;

    for (b=8*sizeof(int)-1; b>=0; b--)
        cout << ((a & (1U<<b)) > 0);
    cout << endl;

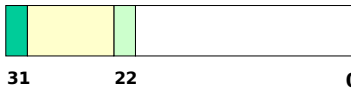
    a = 13;
    for (b=8*sizeof(int)-1; b>=0; b--)
        cout << ((a & (1U<<b)) > 0);
    cout << endl;

    return 0;
}
```

```
11111111111111111111111111111111
000000000000000000000000000000001101
```

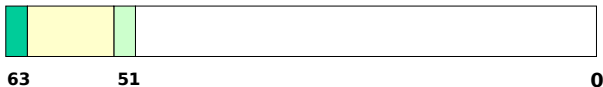
Reprezentacja liczb zmiennoprzecinkowych (IEEE 754)

float

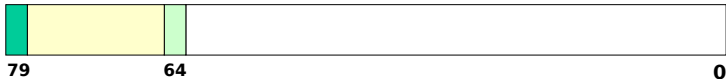


$$\text{liczba} = \pm \text{mantysa} \cdot 2^{\pm \text{wykładnik}}$$

double



long double



- znak
wykładnika



- wykładnik



- znak liczby



- mantysa

Pozostałe operatory

- Operator wyliczeniowy:

`a, b, c;`

`a, b` i `c` są dowolnych typów. Opracowanie wyrażenia jest zgodne z kolejnością (od lewej do prawej). Typowe zastosowanie: wszędzie tam, gdzie nie jest możliwe użycie separatora `;`. Na przykład: `for (i=0, j=0; i < n; i++, j+=2)`

- Operatory wyboru składowych (`.` oraz `->`): omówimy później.
- Operator indeksowania `[]`: omówimy później.
- Operator pobrania adresu `&`: omówimy później.
- Operator dostępu do zmiennej wskazywanej `*`: omówimy później.

Opracowywanie wyrażeń

Opracowywanie wyrażenia to wszystkie czynności (obliczenia i inne) wykonywane podczas przetwarzania wyrażenia.

Kolejność wykonywania działań:

- W przypadku operatorów o różnych priorytetach decyduje priorytet,
np.: $a + b * c \mapsto a + (b * c)$
- W przypadku operatorów dwuargumentowych o identycznych priorytetach decyduje wiązanie:
 - **od lewej do prawej**: $a \circ b \circ c \mapsto (a \circ b) \circ c$.
Dotyczy wszystkich operatorów z wyjątkiem przypisania i operatora warunkowego.
 - **od prawej do lewej**: $a = b = c \mapsto a = (b = c)$.
Dotyczy operatora przypisania i warunkowego.
- Dla szeregu operatorów, kolejność wartościowania (obliczania operandów) nie jest ściśle określona i zależy od implementacji.

Priorytety operatorów

Im wyżej operator znajduje się w tabeli, tym wyższy jego priorytet.

Typ	Operatory	Wiązanie*
Wyrażenia	() [] . -> wyrażenie++ wyrażenie--	L⇒R
Unarne	* & + - ! ~ ++wyrażenie --wyrażenie (rzutowanie) sizeof	R⇒L
Binarne	* / % + - >> << > < <= >= == != & ^ && 	L⇒R
Warunkowy	?:	R⇒L
Przypisania	= += -= *= /= %= >>= <<= &= ^= =	R⇒L
Wyliczeniowy	,	R⇒L

* L⇒R (od lewej do prawej); R⇒L (od prawej do lewej)

Uwaga: używamy nawiasów, aby poprawić czytelność wyrażień.

Kolejność wykonywania obliczeń – punkty sekwencyjne

- Wyrażenia rozdzielone średnikami są wartościowane po kolei.
- Wyrażenia rozdzielone przecinkami są wartościowane po kolei (dotyczy przecinka rozdzielającego definicje zmiennych, nie dotyczy przecinka rozdzielającego argumenty funkcji).
- Wyrażenia w blokach `if(...)`, `switch(...)`, `while(...)` i `for (...; ... ; ...)` są wartościowane w odpowiednim momencie, zgodnie ze specyfikacją działania warunku/pętli.
- Argumenty przekazywane do funkcji są obliczane przed jej wykonaniem, a wartość zwracana (`return`) przed jej zakończeniem.
- Lewy operand operatora `&&` jest zawsze wartościowany przed prawym, a jeśli jego wartość jest zerowa, prawy operand nie jest wartościowany.
- Lewy operand operatora `||` jest zawsze wartościowany przed prawym, a jeśli jego wartość jest niezerowa, prawy operand nie jest wartościowany.
- W operatorze `? :`, warunek jest wartościowany przed odpowiednim (wybrany) spośród następujących po nim wyrażen.

Typowe błędy programisty

- `if ( 0 < a < 5 ) ...` Wyrażenie w nawiasie jest zawsze prawdziwe, ponieważ kompilator traktuje je następująco: $(0 < a) < 5$. Wówczas wyrażenie w nawiasie przyjmuje wartość 0 lub 1, więc otrzymujemy $0 < 5$ lub $1 < 5$, co jest zawsze prawdziwe.
- `if ( i = 5 ) ...` Wyrażenie w nawiasie jest zawsze prawdziwe: następuje przypisanie liczby 5 do zmiennej i, w wyniku czego wartością wyrażenia w nawiasie jest 5 (czyli "prawda").
- `if ( i != 5 ) ...` Wyrażenie jest traktowane jako `i!=(5)`. Czyli, najpierw obliczana jest negacja 5, a następnie rezultat jest przypisywany do zmiennej i.
- `cout << 011;` Instrukcja wypisze na ekran liczbę 9.
- `if ( a || b++ ) ...` Programista nie może liczyć na to, że inkrementacja zmiennej b nastąpi. (Nie nastąpi jeśli $a \neq 0$.)
- `if ( a && b++ ) ...` Programista nie może liczyć na to, że inkrementacja zmiennej b nastąpi. (Nie nastąpi jeśli $a=0$.)
- `if ( 6 & 2 == 2 ) ...` Wyrażenie fałszywe, gdyż jest ewaluowane jako `6 & (2 == 2)` (ze względu na priorytety)

Konwersja arytmetyczna – zasady

W języku C/C++ wyrażenia arytmetyczne mogą obejmować zarówno operandy całkowite, jak i zmiennoprzecinkowe. Konwersja typów następuje przede wszystkim w następujących sytuacjach:

- **konwersja wymuszona przez operator rzutowania bądź sufix przy stałej:**
(int)(a+3.5), (float)x, (unsigned)y, (double)5, 5F, 5U
- **konwersja przy przypisaniu:** double x = -7.7; int a = x; Uwaga: konwersja typu zmiennoprzecinkowego na całkowity zawsze odbywa się pośrednio poprzez typ long, poprzez odrzucenie części liczby po kropce (w powyższym przykładzie: a = -7)
- **konwersja przy wykonywaniu operacji arytmetycznych:**
 - **long double** o **wyrażenie** – obydwa operandy traktowane jako long double
 - w przeciwnym razie: **double** o **wyrażenie** – obydwa jako double
 - w przeciwnym razie: **float** o **wyrażenie** – obydwa jako float
 - w przeciwnym razie: **unsigned long** o **wyrażenie** – obydwa jako unsigned long
 - w przeciwnym razie: **unsigned int** o **long** – obydwa jako unsigned long
 - w przeciwnym razie: **long** o **wyrażenie** – obydwa jako long
 - w przeciwnym razie: **unsigned int** o **wyrażenie** – obydwa jako unsigned int
 - w każdym innym przypadku: obydwa operandy traktowane jako int

Konwersja arytmetyczna – przykłady

```
#include <iostream>
using namespace std;
/* Program ilustrujący potencjalne
   kłopoty związane z konwersją typów */
int main() {
    int a = -1;
    unsigned int b = 1;
    short c=100;
    long long l;
    char znak = '0';

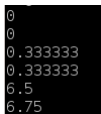
    cout << (unsigned) a << endl;
    cout << (a < b) << endl;      // niejawna konwersja a do typu unsigned int
    cout << a+1 << endl;          // wszystko dobrze (oba operandy typu int)
    cout << a+ (unsigned int)1 << endl; // wszystko dobrze... przypadkiem
    a = -2;
    cout << a+ (unsigned int)1 << endl; // niejawna konwersja a do unsigned int
    cout << c*c*c*c << endl; // wszystko dobrze: mnożenie po konwersji do int
    l = ((long long)1)<<35; // wszystko dobrze
    cout << l << endl;
    l = 1<<35; // możliwy błąd gdy int jest 32-bit. Kompilator może ostrzegać!
    cout << l << endl;
    cout << znak << endl; // wypisujemy znak
    cout << (int) znak << endl; // oraz jego kod ASCII
    return 0;
}
```

```
deren@linux-nb6o:~> g++ -o w04p02 w04p02.cpp
w04p02.cpp: In function 'int main()':
w04p02.cpp:21:10: warning: left shift count >
= width of type [enabled by default]
deren@linux-nb6o:~> ./w04p02
4294967295
0
0
0
4294967295
1000000000
34359738368
0
0
48
```

Konwersja arytmetyczna – przykłady

```
#include <iostream>
using namespace std;
/* Program ilustrujący potencjalne
   kłopoty związane z konwersją typów
   (zaokrąglenia) */
int main() {
    double d = 1/3; // dzielenie całkowitoliczbowe!
    int i = 1/3;

    cout << i << endl;
    cout << d << endl;
    d = 1/3.0; // operandy zmiennoprzecinkowe
    cout << d << endl;
    d = (1.0F)/3; // operandy zmiennoprzecinkowe
    cout << d << endl; // j.w.
    // Uwaga: dwa poniższe wyrażenia dają
    // inne wyniki. Powód: opracowywanie
    // wyrażenia "od lewej do prawej"
    d = 27/2/2.0;
    cout << d << endl;
    d = 27.0/2/2;
    cout << d << endl;
    return 0;
}
```



```
0
0
0.333333
0.333333
6.5
6.75
```

Podstawy Programowania

Wykład nr 5: Tablice

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Czym jest tablica?

- Typy proste służą do przechowywania pojedynczej wartości. Czasami zachodzi konieczność przechowania wielu wartości tego samego typu. Do tego celu wykorzystujemy tablicę.
- **Tablica** jest sekwencją elementów tego samego typu, które zajmują ciągły obszar w pamięci operacyjnej.
- Tablic używamy więc w sytuacji, gdy mamy do czynienia z sekwencją danych (ciąg liczb, ciąg znaków, itp.)
- Deklaracja tablicy o nazwie **t** zawierającej **X** elementów typu **typ**:

```
typ t[X];
```

- odwołanie się do elementu numer **i** w powyższej tablicy: **t[i]**
- Tablice można także tworzyć dynamicznie (w trakcie działania programu) oraz odwoływać się do poszczególnych elementów “poprzez wskaźnik”. O tych metodach powiemy później.

Odwołania do elementów tablicy

```
int t[5];
```

t[0] t[1] t[2] t[3] t[4]

- Numeracja elementów tablicy rozpoczyna się od **zera**.
- Element **t[5]** nie został zaalokowany w powyższym przykładzie.
- W języku C nie następuje automatyczne sprawdzanie zakresów tablic. To programista jest odpowiedzialny za to, aby kontrolować zakresy indeksów.
- Próba odwołanie się do **t[5]** nie zostanie zasygnalizowana przez kompilator jako błąd.
- Wpisanie wartości do nieistniejącego elementu **t[5]** spowoduje pisanie do obszaru pamięci znajdującego się bezpośrednio za tablicą. Może to spowodować "uszkodzenie" wartości zmiennej, która się tam znajduje.

Inicjalizacja wartości tablicy

- Sposób pierwszy: podajemy rozmiar tablicy i wszystkie jej elementy:

```
int t[5] = {1,3,5,7,9};
```

Skutkuje to: $t[0]=1$, $t[1]=3$, $t[2]=5$, $t[3]=7$, $t[4]=9$.

- Sposób drugi: nie podajemy rozmiaru tablicy, a jedynie wszystkie jej elementy:

```
int t[] = {1,3,5,7,9};
```

Skutkuje to: $t[0]=1$, $t[1]=3$, $t[2]=5$, $t[3]=7$, $t[4]=9$. Kompilator sam przeliczy ile wartości podano w nawiasach klamrowych i na tej podstawie dobiera odpowiedni rozmiar tablicy.

- Sposób trzeci: podajemy rozmiar tablicy i część jej elementów:

```
int t[5] = {1,3,5};
```

Skutkuje to: $t[0]=1$, $t[1]=3$, $t[2]=5$, $t[3]=0$, $t[4]=0$, gdyż brakujące elementy są inicjowane na wartość 0.

(Ciekawostka: w standardzie C99: `int t[]={ [0]=1, [2]=5, [4]=9 }.`)

Dyrektywa #define – wprowadzenie

#define NAZWA WARTOSC

- Dyrektywy są przetwarzane przez preprocesor przed przystąpieniem do kompilacji.
- Przetwarzanie polega na zastąpieniu każdego wystąpienia w kodzie literału **NAZWA** przez **WARTOSC**.
- **WARTOSC** może być stałą lub bardziej skomplikowanym wyrażeniem.
- Preprocesor nie wykonuje analizy składniowej wyrażeń – jedynie dokonuje podstawień. Ewentualne błędy są wskazywane podczas kompilacji.
- Ogólnie przyjęta konwencja: identyfikatory (NAZWA) piszemy dużymi literami.
- Dobrym stylem programisty jest używanie dyrektyw #define do nadawania identyfikatorów stałym używanym w programie, np.:

```
#define LICZBA_MIESIECY 12
```

```
#define KOLOR_TLA 17
```

```
#define ROZMIAR_TABLICY 1000
```

```
#define SZEROKOSC_EKRANU 1200
```

Dzięki temu zmiana wartości (jeśli jest taka potrzeba) następuje tylko w jednym miejscu programu (przy dyrektywie), a nie w każdym jej wystąpieniu w kodzie.

Przykładowy problem

Założmy, że potrzebujemy w programie zdefiniować tablicę, która zawiera liczby o określonej własności. Nazwijmy je *liczbami szczególnymi*. Na razie jest ich pięć, lecz pisząc program mamy na uwadze, że na dalszym etapie być może będziemy musieli rozszerzyć tę tablicę o więcej takich liczb.

```
#include <iostream>
using namespace std;

/* Aby dodać nową liczbę do tablicy i oczekiwać wypisania
wszystkich liczb na ekran należy:
    1) dopisać liczbę do tablicy
    2) zwiększyć rozmiar tablicy o 1
    3) zwiększyć stałą w warunku  $i < 5$ 

Uwaga: rozszerzanie staje się kłopotliwe w przypadku, gdy
miejsce wymagających zmiany 5 na 6 jest dużo więcej.
Takię podejście jest przykładem złego stylu programowania.
*/
int main() {
    int liczby_szczegolne[5] = { 7, 13, 16, 21, 14 }, i;

    for ( i=0; i < 5; i++ )
        cout << liczby_szczegolne[i] << endl;
    return 0;
}
```

Przykładowy problem c.d.

Poniższy program eliminuje wcześniejszy problem: dodanie elementu w tablicy wymaga **tylko** uaktualnienia wartości stałej **ROZMIAR**.

```
#include <iostream>
using namespace std;

#define ROZMIAR 5
/* Aby dodać nową liczbę do tablicy i oczekiwać wypisania
wszystkich liczb na ekran należy:
    1) dopisać liczbę do tablicy
    2) zmienić deklarację ROZMIAR (z 5 na 6)

Uwaga: Jeśli w kodzie będziemy używać zmiennej rozmiar zamiast
stałej numerycznej 5, to mamy gwarancję, że te dwie
czynności wystarczą przy rozszerzeniu tablicy
Pytanie: czy da się jeszcze wygodniej?
*/
int main() {
    int liczby_szczególne[ROZMIAR] = { 7, 13, 16, 21, 14 }, i;

    for ( i=0; i < ROZMIAR; i++ )
        cout << liczby_szczególne[i] << endl;
    return 0;
}
```

Przykładowy problem c.d.

- W celu dalszego usprawnienia programu z pomocą przychodzi operator `sizeof`.
- Oprócz omawianej dotychczas składni (`sizeof(typ)`) operator można użyć następująco: `sizeof x`, gdzie `x` jest wyrażeniem. Wówczas, wartością `sizeof x` (gdzie `x` jest wyrażeniem) jest liczba bajtów obiektu takiego typu jak `x`.
- Jeśli `x` jest tablicą, to `sizeof x` zwraca liczbę bajtów (nie elementów!) tablicy.
- Jeśli `x` jest tablicą, to `(sizeof x)/(sizeof x[0])` da nam liczbę elementów tablicy `x`.

```
#include <iostream>
using namespace std;

/* Aby dodać nową liczbę do tablicy i oczekiwać wypisania
   wszystkich liczb na ekran należy jedynie...
   dopisać liczbę do tablicy
*/
int main() {
    int liczby_szczególne[] = { 7, 13, 16, 21, 14 }, i;

    for (i=0; i<(sizeof liczby_szczególne)/(sizeof liczby_szczególne[0]); i++)
        cout << liczby_szczególne[i] << endl;
    return 0;
}
```

7
13
16
21
14

Inicjalizacja tablic typu char

- Oprócz wcześniej wymienionych metod inicjalizacji tablic, możemy wykorzystać fakt, że stała znakowa umieszczona w cudzysłowach (") jest traktowana jako tablica znaków, której ostatnim elementem jest znak `'\0'`. Ten ostatni element jest umieszczany dodatkowo za ostatnim znakiem stałej znakowej.
- Poniższe dwie definicje są równoważne:

```
char s[10] = { "napis" };
```

```
char s[10] = { 'n', 'a', 'p', 'i', 's', '\0' };
```

W pierwszym przypadku nawiasy `{ }` można pominąć. W obu przypadkach, pozostałe elementy, począwszy od indeksu numer 6, są inicjalizowane wartością 0:

0	1	2	3	4	5	6	7	8	9
'n'	'a'	'p'	'i'	's'	'\0'	0	0	0	0

- Ile elementów ma tablica: `char s[] = { "Halo!" };`? Odp.: 6 (`s[5]` jest równe `'\0'`).

Łańcuchy znaków

Powody, dla których należy pamiętać o znaku `'\0'` kończącym napis:

- Funkcje biblioteczne zakładają, że `'\0'` jest zakończeniem napisu. Na przykład, wypisywanie napisu na ekran monitora odbywa się poprzez wypisywanie kolejnych znaków, aż do napotkania `'\0'`. W przypadku gdy znak ten nie wystąpi w tablicy, funkcja wypisująca napis na ekran będzie kontynuować swoją pracę, odczytując kolejne bajty w pamięci operacyjnej znajdującej się poza tablicą, i traktując je jako kody ASCII będzie wypisywać znaki na ekran.
- Na podstawie zawartości tablicy (tzn. na podstawie analizy jej kolejnych elementów), wystąpienie znaku `'\0'` jest jedynym sposobem pozwalającym na określenie końca napisu.

Łańcuchy znaków – przykładowy program

```
#include <iostream>
using namespace std;

/* Przykłady manipulacji napisami */
int main() {
    char s[256] = { "Jakis ciekawy napis" };
    int i=0;

    cout << s << endl;
    cout << sizeof s << endl; // zwraca rozmiar tablicy, nie napisu!
    s[13] = '\0';
    cout << s << endl; // pozostałe znaki nadal są w tablicy,
                        // lecz nie są wypisywane

    s[14] = 'p'; // te dwie zmiany w tablicy następują "poza" pierwszym
    s[15] = 'o'; // wystąpieniem znaku '\0' w tablicy s, więc choć
    cout << s << endl; // zmieniają zawartość tablicy, nie wpływają na to,
                        // co się pojawi na ekranie

    s[13] = '\t';
    cout << s << endl;

    cin >> s;
    while ( s[i] != '\0' ) // obliczymy ile znaków ma słowo podane
        i++;              // przez użytkownika
    cout << i << endl;
    return 0;
}
```

```
Jakis ciekawy napis
256
Jakis ciekawy
Jakis ciekawy
Jakis ciekawy   popis
Abrakadabra
11
```

Tablice wielowymiarowe

Przykład deklaracji tablicy dwuwymiarowej o 4 wierszach i 3 kolumnach:

```
char t[4][3];
```

Do elementów tablicy odwołujemy się następująco:

t[0][0]	t[0][1]	t[0][2]
t[1][0]	t[1][1]	t[1][2]
t[2][0]	t[2][1]	t[2][2]
t[3][0]	t[3][1]	t[3][2]

Inicjalizacja tablicy dwuwymiarowej:

```
char t[4][3] = { {1,2,3}, {4,5,6}, {7,8,9}, {10,11,12} };
```

Tablice dwuwymiarowe – przykład

- Napiszmy program sprawdzający czy podana na wejściu 2-wymiarowa tablica o wymiarach $n \times n$ jest symetryczna. Tablicę nazywamy symetryczną, gdy element znajdujący się w i -tym wierszu i j -tej kolumnie jest taki sam jak element znajdujący się w j -tym wierszu i i -tej kolumnie.
- Na przykład: tablica A jest symetryczna, natomiast tablica B taka nie jest:

$$A = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 4 & 5 & 6 \\ 2 & 5 & 7 & 8 \\ 3 & 6 & 8 & 9 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 4 & 5 & 6 \\ 2 & 5 & 7 & 8 \\ 3 & 7 & 8 & 9 \end{bmatrix}$$

- Program będzie działał na tablicy o ustalonym rozmiarze, np. 3 lub 4, ale aby pozostawić łatwą możliwość zmiany rozmiaru, skorzystamy z dyrektywy `#define`.

Tablice dwuwymiarowe – przykład

```
#include <iostream>
using namespace std;
#define ROZMIAR 2

/* Program sprawdzający czy podana na wejściu tablica jest symetryczna */
int main() {
    int i, j, tablica[ROZMIAR][ROZMIAR], symetria;

    for ( i=0; i < ROZMIAR; i++ )
        for ( j=0; j < ROZMIAR; j++ ) {
            cout << "Podaj wartość elementu X[" << i << "][" << j << "] = ";
            cin >> tablica[i][j];
        }

    for ( symetria=1, i=0; i < ROZMIAR; i++ )
        for ( j=0; j < i; j++ )
            if ( tablica[i][j] != tablica[j][i] )
                symetria = 0;

    cout << "Tablica " << (symetria==1 ? "" : "nie ") << "jest symetryczna\n";
    return 0;
}
```

```
5
Podaj wartość elementu X[0][0] = 0
Podaj wartość elementu X[0][1] = 1
Podaj wartość elementu X[1][0] = 2
Podaj wartość elementu X[1][1] = 3
Tablica nie jest symetryczna
```

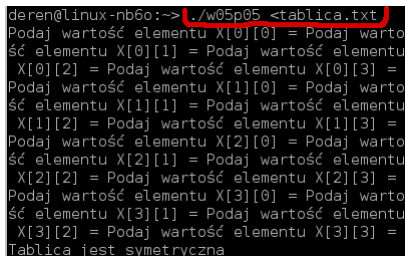
Usprawnienie testowania programów

- Założmy, że chcemy poprzedni program przetestować dla nieco większych tablic, np. rozmiaru 4×4 . (Czyli zmieniamy linijkę `#define ROZMIAR 4`.)
- Wielokrotne uruchamianie programu w procesie testowania staje się procesem żmudnym i mało efektywnym, gdy za każdym razem jesteśmy zmuszeni do wpisywania 16 elementów tablicy.
- Z pomocą przyjdzie nam mechanizm przekierowania wejścia w konsoli.
- Tworzymy plik tekstowy, np. o nazwie `tablica.txt`, o treści jak wyżej (każda jego linia kończy się znakiem enter).



0	1	2	3
1	4	5	6
2	5	7	8
3	6	8	9

- W linii poleceń korzystamy z przekierowania wejścia. Środowisko uruchomieniowe programu (konsola) dba o to, aby program nie dostrzegł żadnej różnicy w sytuacjach, gdy dane wejściowe pochodzą z klawiatury, czy są przekierowane z pliku.
- Efekty uboczne! (patrz obok, gdzie `w05p05` jest nazwą pliku wykonywalnego).
- Aby “oddzielić” wejścia programu od wyjścia: `./w05p05 <tablica.txt >wyjscie.txt` (To, co wypisuje program zostanie umieszczone w pliku `wyjscie.txt`.)



```
dereh@linux-nb6o:~$ ./w05p05 <tablica.txt
Podaj wartość elementu X[0][0] = Podaj wartość
Podaj wartość elementu X[0][1] = Podaj wartość elementu
Podaj wartość elementu X[0][2] = Podaj wartość elementu X[0][3] =
Podaj wartość elementu X[1][0] = Podaj wartość
Podaj wartość elementu X[1][1] = Podaj wartość elementu
Podaj wartość elementu X[1][2] = Podaj wartość elementu X[1][3] =
Podaj wartość elementu X[2][0] = Podaj wartość
Podaj wartość elementu X[2][1] = Podaj wartość elementu
Podaj wartość elementu X[2][2] = Podaj wartość elementu X[2][3] =
Podaj wartość elementu X[3][0] = Podaj wartość
Podaj wartość elementu X[3][1] = Podaj wartość elementu
Podaj wartość elementu X[3][2] = Podaj wartość elementu X[3][3] =
Tablica jest symetryczna
```

Podstawy Programowania

Wykład nr 6: Funkcje

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Dlaczego funkcje są potrzebne?

- Podczas pisania programu często zachodzi potrzeba wielokrotnego wykonywania tych samych obliczeń (tzn. wykonywania tych sekwencji instrukcji) w różnych sytuacjach (zazwyczaj dla różnych danych). Przykładami mogą być: wypisanie komunikatu informującego o stanie programu, znalezienie najmniejszego elementu w tablicy, posortowanie tablicy, wyznaczenie rozwiązań równania kwadratowego.
- W obrębie funkcji można zamknąć operacje wykonywane dla zadanych parametrów wejściowych, np. ciąg liczb, współczynniki równania kwadratowego a , b , c .
- W praktyce każdy (większy) program jest zbudowany z wielu funkcji (dobry styl programowania, wiele krótkich i czytelnych funkcji).
 - poprawia to przejrzystość i czytelność programu,
 - zapewnia oszczędność czasu (funkcja raz napisana może być wielokrotnie wykorzystana),
 - ułatwia dokonywanie zmian w programie: jeśli daną, wielokrotnie powtarzaną czynność należy zmodyfikować, to czynimy tą modyfikację w jednym tylko miejscu, gdy jest ona wyodrębniona jako funkcja,
 - pozwala zaoszczędzić pamięć: funkcja jest umieszczona w jednym miejscu w pamięci podczas wykonywania programu.

Deklaracja a definicja funkcji

- **Deklaracja funkcji:** informuje kompilator o tym, że określona funkcja może (ale nie musi) zostać użyta w programie. Deklaracja zawiera informację o nazwie funkcji, typie zwracanej wartości oraz typach argumentów. Deklaracja nie dostarcza kodu funkcji.

```
int min( int a, int b);
```

- **Definicja funkcji:** informuje o tym co funkcja robi. W odróżnieniu od deklaracji, definicja skutkuje przydzieleniem przez kompilator pamięci na kod funkcji.

```
int min( int a, int b) {  
    if ( a < b ) return a;  
    else return b;  
}
```

Przykład użycia funkcji – program obliczający współczynnik dwumianu

- Zadanie polega na napisaniu programu obliczającego wartość $\binom{n}{k}$ dla podanych przez użytkownika wartości n i k , gdzie $n \geq k$.
- Postanawiamy skorzystać ze wzoru:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

- Obserwujemy, że program będzie wymagał trzykrotnego obliczenia silni dla różnych argumentów.
- Aby nie pisać trzykrotnie identycznego kodu (a jedynie operującego na innych zmiennych), decydujemy się na napisanie funkcji obliczającej silnię z zadanego argumentu.
- Silnia nawet dla niewielkich argumentów przyjmuje znaczące wartości (np. $20! = 2432902008176640000$), więc aby zwiększyć liczbę argumentów, dla których nie nastąpi efekt przepełnienia typu, wartości zwracane przez naszą funkcję będą typu `long int`.

Przykład użycia funkcji – program obliczający współczynnik dwumianu

```
#include <iostream>
using namespace std;

long long silnia( int );

/* Program obliczający współczynnik dwumianowy z liczb
n i k podanych przez użytkownika.
Uwaga: nawet dla małych wartości n może nastąpić przepełnienie. */
int main() {
    int n, k;
    cin >> n;
    cin >> k;
    cout << silnia(n)/( silnia(k)*silnia(n-k) ) << endl;
    return 0;
}

long long silnia( int n ) {
    long long s = 1;

    for (int i=1; i<=n; i++)
        s *= i;
    return s;
}
```

Przykład użycia funkcji – zadanie domowe

Napisz program obliczający wartość e^x , gdzie e jest liczbą Eulera ($e \approx 2.718$) a x jest liczbą rzeczywistą.

- Język C nie posiada operatora potęgowania w swoim asortymencie.
- W celu obliczenia e^x moglibyśmy użyć odpowiedniej funkcji matematycznej z biblioteki `math.h`. Elementem dobrego stylu programowania jest korzystanie z gotowych funkcji bibliotecznych gdy tylko jest to możliwe. W tym wypadku jednak świadomie z tego zrezygnujemy.
- Nie możemy obliczyć wartości e^x mnożąc e przez siebie x razy, gdyż x nie musi być liczbą całkowitą. Z pomocą przychodzi nam wzór:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

Oczywiście, obliczanie nieskończonej sumy nie jest możliwe, więc ograniczymy się do przybliżenia wartości e^x poprzez sumowanie kilku początkowych wartości powyższego szeregu, np.

$$e^x \approx \sum_{n=0}^7 \frac{x^n}{n!} = \frac{x^0}{0!} + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!}.$$

Przykład użycia funkcji – zadanie domowe

Zanim zaczniemy pisać program, warto dokonać szeregu przemyśleń, które ułatwią kodowanie.

- Nie wiemy ile początkowych wyrazów szeregu powinniśmy brać pod uwagę (8 we wcześniejszym wzorze). Nasz plan jest taki, aby sprawdzić to eksperymentalnie. Aby program był łatwy do modyfikacji, spróbujemy liczbę wyrazów określać za pomocą stałej. Na początku programu umieścimy więc dyrektywę

```
#define LICZBA_WYRAZOW 8
```

i dalej w kodzie posługujemy się tym identyfikatorem zamiast stałą liczbową.

W konsekwencji, jeśli postanowimy, że program powinien uwzględnić początkowych 10 wyrazów, zmienimy to zmieniający 8 na 10 tylko w dyrektywie. Powinna to być jedyna zmiana wymagana w tym przypadku!

- Zauważamy, że pewne obliczenia się powtarzają: obliczanie $n!$ oraz x^n . Napiszmy więc funkcje, które wykonują te obliczenia. Jakie typy powinny mieć parametry i zwracane wartości?
- Samo obliczenie e^x też warto wyodrębnić jako osobną funkcję pobierającą na wejściu x (typu `double`) oraz zwracającą wartość e^x (również typu `double`).
- Uwaga dla zainteresowanych: program napisany w ten sposób jest podatny na błędy przepiętnień i zaokrągleń. Metody jego udoskonalania wykraczają poza zakres tego wykładu.

Zasięg zmiennych

- Obiekt zdefiniowany na zewnątrz wszystkich funkcji ma **zasięg globalny**.
 - Zmienne globalne oraz zmienne z modyfikatorem **static** umieszczane są w normalnym obszarze pamięci (static), który przed uruchomieniem programu jest zerowany, a wartości zmiennych są zachowywane pomiędzy kolejnymi uruchomieniami funkcji.
- **Zmienne lokalne** funkcji (oraz **argumenty** przekazywane do funkcji) są zmiennymi **automatycznymi** (auto).
 - W momencie, gdy kończymy blok, w którym zmienne lokalne zostały powołane do życia, automatycznie przestają one istnieć.
 - Obiekty automatyczne komputer przechowuje na stosie.
 - Należy pamiętać, że zmienne automatyczne nie są zerowane w chwili utworzenia.
 - Jeśli nie zainicjowaliśmy ich jakąś wartością, to przechowują one **wartości przypadkowe**.

```
#include <iostream>
using namespace std;

int x;
void f( void ) {
    int i;
}

int main() {
    int a = 1;
    {
        int b = 2;
        cout << x; // 0 (globalne x)
    }
    cout << x; // 0 (globalne x)
    {
        int b = 3;
        int x = 4;
        int a = 5;
        {
            int d = 6;
            int x = 7;
            cout << a; // 5
            cout << b; // 3
            cout << x; // 7 (def. int x=7)
        } // przestania obie poprzednie
        cout << b; // 3
        cout << a; // 5
        cout << x; // 4
        cout << d; // Błąd kompilacji!
    }
    return 0;
}
```

Do jakich zmiennych ma dostęp funkcja w języku C?

- Z poziomu funkcji uzyskujemy **jawny dostęp** do zmiennych globalnych, zmiennych lokalnych oraz argumentów bieżącego wykonania funkcji (**argumenty są traktowane tak samo, jak zmienne lokalne!**)
- Niejawny dostęp następuje wówczas, gdy funkcja posiada adres pamięci (wskaźnik) innej zmiennej. Dzięki temu, możliwa jest zmiana wartości obszaru pamięci pod tym adresem, a przez to zmiana wartości zmiennej tam umieszczonej.
- Nie istnieje żadna forma ochrony pamięci: korzystając ze wskaźników można odczytać bądź nadpisać dowolny obszar pamięci.
 - działania zamierzone: modyfikacja tablic, modyfikacja zmiennych lokalnych innych funkcji (poprzez wskaźniki do nich)
 - błędne użycie może prowadzić do niezamierzonego działania programu: uszkodzenie stosu wykonania, wkroczenie na „segment kodu” lub odwoływanie się do obszaru pamięci, który nie należy do programu (reakcja na taką sytuację może zależeć od systemu operacyjnego).
- Funkcja nie posiada jawnego dostępu do zmiennych lokalnych innych funkcji.
- Przesłanianie deklaracji zmiennych: w różnych blokach kodu można deklarować zmienne o tej samej nazwie. Odwołanie do zmiennej poprzez jej nazwę odnosi się do zmiennej o tej nazwie zadeklarowanej w najbardziej wewnętrznym bloku — patrz poprzedni przykład.

Inicjalizacja zmiennych i zmienne static – przykład

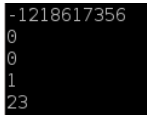
```
#include <iostream>
using namespace std;

int x;

/* Funkcja zwraca liczbę całkowitą, która jest równa liczbie wywołań
funkcji od czasu uruchomienia programu.
Uwaga: licznik nie jest zabezpieczony przed przepiętnieniem. */
int f( void ) {
    static int licz_wywołania; // zmienna ta istnieje i zachowuje swoją
    // wartość pomiędzy kolejnymi wywołaniami funkcji
    return ++licz_wywołania;
}

int main() {
    int a;
    static int b;

    cout << a << endl; // na ekran zostanie wypisana losowa wartość
    cout << b << endl; // wypisuje 0 (zmienne static inicjowane są zerami)
    cout << x << endl; // wypisuje 0 (zmienne globalne inicjowane są zerami)
    cout << f() << endl;
    f();
    a = f() + f()*f(); // Uwaga: tutaj mamy do czynienia z efektem ubocznym:
    // wiadomo, że 3 powyższe wywołania funkcji f() zwrócą wartości 3,4,5,
    // lecz nie wiadomo które wywołanie zwróci którą wartość!!
    cout << a << endl;
    return 0;
}
```



```
-1218617356
0
0
1
23
```

Przekazywanie parametrów

- W języku C występuje **tylko** tzw. przekazywanie parametrów przez wartość, które polega na tym, że każdy argument funkcji jest kopią przekazywanego parametru.
- W przypadku tablicy identyfikator jest adresem tej tablicy. Jeśli więc argumentem funkcji jest tablica, to funkcja otrzymuje kopię adresu przekazywanej tablicy, a nie kopię całej tablicy. Stąd efekt polegający na tym, że zmiana wartości elementu tablicy wewnątrz funkcji powoduje zmianę wartości tablicy przekazanej do funkcji – jest to ta sama tablica (patrz przykład obok).

```
#include <iostream>
using namespace std;
```

```
void f( int x ) {
    x = 1;
}
```

/ Funkcja modyfikuje wartość x[1]. Uwaga: programista musi zadbać, aby przekazana na wejście tablica miała rozmiar co najmniej 2 */*

```
void g( int x[] ) {
    x[1] = 1;
}
```

```
int main() {
    int x=0, t[] = { 0, 0, 0, 0, 0, 0, 0, 0 };

```

```
    f( x );
    cout << x << endl; // wypisujemy 0

```

```
    g( t );
    // ponizej wypisujemy 0,1,0,...
    for ( int i=0; i < (sizeof t)/(sizeof t[0]); i++ )
        cout << t[i] << ' ';
    cout << endl;
    return 0;
}
```

```
0
0,1,0,0,0,0,0,0,
```

Zwracanie wartości

- Słowo kluczowe **return** z parametrem, odpowiadającym typowi zwracanemu określonego dla funkcji, powoduje zakończenie wykonania funkcji i zwrócenie wartości do funkcji wołającej.
- Typ wyrażenie będącego parametrem instrukcji **return** musi umożliwiać rzutowanie do typu zwracanego przez funkcję.
- Wartość zwracana jest zawsze kopiowana (podobnie, jak przy przekazywaniu argumentów).
- Operacja return powinna się wykonać dla każdej możliwej ścieżki wykonania funkcji w przypadku funkcji zwracającej typ inny niż **void**.
- W funkcji o typie zwracanym **void**, instrukcja **return** jest opcjonalna. W takich przypadkach funkcja kończy się wraz z zakończeniem realizacji jej bloku instrukcji.

Biblioteka standardowa – ważniejsze pliki nagłówkowe

- **assert.h** – diagnozowanie programów
- **ctype.h** – klasyfikacja znaków
- **errno.h** – zmienne przechowujące informacje o błędach
- **math.h** – podstawowe funkcje matematyczne
- **signal.h** – obsługa zdarzeń wyjątkowych
- **stdio.h** – obsługa wejścia/wyjścia
- **stdlib.h** – funkcje ogólnego przeznaczenia, włączając alokację pamięci, konwersje, i inne
- **string.h** – funkcje manipulacji napisami
- **time.h** – obsługa daty i czasu

Przykład 1: wyszukanie zadanego elementu w tablicy

Założmy, że chcemy napisać funkcję, która stwierdza czy w podanej tablicy znajduje się zadany element. Podejmujemy szereg decyzji na etapie projektowania takiego fragmentu kodu:

- Funkcja taka mogłaby zwracać wartości **0** oraz **1** wskazujące na to czy element znajduje się w tablicy czy nie. Aby nasza funkcja była bardziej ogólna i mogła być przydatna w szerszej gamie sytuacji, postanawiamy, że będzie ona zwracać numer indeksu, pod którym znajduje się szukany element.
- Musimy mieć możliwość zakomunikowania w sytuacji braku szukanego elementu – wówczas nasza funkcja zwróci **-1**.
- Wśród argumentów funkcji będzie oczywiście tablica (nazwijmy ten argument **tablica**) i poszukiwany element (który nazwiemy **element**). Programista piszący funkcję musi jednak znać rozmiar tablicy. Mamy tutaj dwie główne opcje do wyboru:
 - rozmiar jest zadeklarowany na etapie pisania programu, tzn. deklaracja funkcji byłaby następująca: `int szukaj( int tablica[ROZMIAR], int element );`, gdzie **ROZMIAR** jest etykietą zdefiniowaną przez dyrektywę `#define`. Wada: można używać funkcji tylko do tablic takiego rozmiaru (w programie możemy mieć tablice różnych długości).
 - rozmiar jest również parametrem funkcji. Wówczas deklaracja mogłaby wyglądać tak: `int szukaj( int tablica[], int rozmiar, int element );`
To rozwiązanie nie ma poprzedniej wady, więc je wybieramy.
- Funkcję opatrujemy stosownym komentarzem, który informuje co funkcja robi i jakie jest znaczenie poszczególnych parametrów.

Przykład 1: wyszukanie zadanego elementu w tablicy

```
#include <iostream>
using namespace std;

/* Funkcja szuka liczby o wartości element w tablicy t.
   Parametry:
   1) tablica
   2) n – liczba elementów tablicy t
   3) szukany element
   Zwracana wartość: liczba i,  $0 \leq i < n$ , taka że  $t[i] == \text{element}$ 
   lub -1 jeśli takie i nie istnieje (tzn. gdy liczby element
   nie ma w tablicy t */
int szukaj( const int t[], int n, int element ) {
    for ( int i=0; i < n; i++ ) {
        if ( t[i] == element )
            return i;
    }
    return -1;
}

// W celu przetestowania powyższej funkcji piszemy prosty program testowy:
int main() {
    int t[] = { 0, 0, 0, 4, 0, 9, 0, 4, 5, 3, 4, 5, 6 }, podany;

    cout << "Podaj element, który chcesz znaleźć: ";
    cin >> podany;
    cout << szukaj( t, (sizeof t)/(sizeof t[0]), podany ) << endl;
    return 0;
}
```

Przykład 2: porządkowanie liczb w tablicy

Założmy, że chcemy napisać funkcję, która porządkuje (niemalejąco) elementy podanej na wejściu tablicy. Przed napisaniem kodu dokonujemy szeregu przemyśleń:

- Rozpoczniemy od zastanowienia się w jaki sposób takie zadanie należy wykonać (ta kwestia była dość naturalna w poprzednim przypadku, więc jej wprost nie poruszyliśmy): jeśli tablica ma n elementów, to jeśli potrafilibyśmy znaleźć największy z nich i zamienić ten największy z ostatnim (czyli n -tym) elementem w tablicy, to ten największy element będzie już na swoim właściwym miejscu. Wówczas, wśród pozostałych $n-1$ elementów znajdziemy największy i umieścimy go na przedostatnim ($(n-1)$ -szym miejscu) itd.
- Z powyższego wnioskujemy, że potrzebne są nam następujące fragmenty kodu:
 - Funkcja wyszukująca największy element (będzie ona zwracać indeks największego elementu, a na wejściu pobierać tablicę oraz liczbę jej elementów). Przyjmijmy następującą jej deklarację: `int najwiekszy( int t[], int rozmiar );`
 - Funkcja zamieniająca elementy tablicy o wskazanych indeksach. Funkcja nie musi niczego zwracać. Na wejście funkcja dostanie tablicę oraz indeksy elementów do zamiany. Zauważmy, że rozmiar tablicy nie jest w tej funkcji do niczego potrzebny. Deklaracja: `void zamien(int t[], int indeks1, int indeks2 );`
 - Funkcja główna używająca powyższych do porządkowania tablicy. Funkcja nie zwraca wartości, a na wejściu otrzyma tablicę oraz jej rozmiar. Mamy więc deklarację: `void porzadkuj_tablice( int t[], int n );`

Przykład 2: porządkowanie liczb w tablicy

```
#include <iostream>
using namespace std;

int najwiekszy( int t[], int rozmiar ) {    // Funkcja zwraca indeks
    int indeks = 0, i;                     // w tablicy t, pod którym
    for ( i=1; i < rozmiar; i++ )          // znajduje się jej
        if ( t[i] > t[indeks] )             // największy element.
            indeks = i;
    return indeks;
}

void zamien(int t[], int indeks1, int indeks2 ) {
    int pomocnicza = t[indeks1];           // Funkcja dokonuje zamiany elementów
    t[indeks1] = t[indeks2];               // pod indeksami indeks1 i indeks2
    t[indeks2] = pomocnicza;               // w tablicy t
}

void porzadkuj_tablice( int t[], int n ) { // Funkcja porządkuje
    for ( int i=n-1; i >0; i-- )           // niemalejąco liczby
        zamien( t, i, najwiekszy( t, i+1 ) ); // w talicy t o rozmiarze n
}

int main() { // Dołączamy prosty program testowy.
    int t[] = { 0, 41, 0, 4, 0, 9, 0, 4, 5, 3, 4, 5 , 6, 0, -3, -3, -1, 20 };

    porzadkuj_tablice( t, (sizeof t)/(sizeof t[0]) );
    for ( int i=0; i < (sizeof t)/(sizeof t[0]); i++ )
        cout << t[i] << endl;
    return 0;
}
D.Dereniowski (KAiMS,PG)
```

Podstawy Programowania

Wykład nr 7: Wprowadzenie do wskaźników

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Czym jest wskaźnik?

- Z punktu widzenia programisty, pamięć w komputerze jest zorganizowana w postaci jednego bloku pamięci, w którym komórki indeksowane są liczbami naturalnymi.
- Każda zmienna, w trakcie działania programu, zajmuje określoną liczbę komórek w pamięci operacyjnej.
- Wyjątek od powyższej zasady jest możliwy (aczkolwiek niezauważalny dla programisty): jeśli w programie nie następuje jawne odwołanie do adresu zmiennej, to zmienna ta, jeśli tak postanowi kompilator, może być przechowywana wyłącznie w rejestrach procesora.
- **Wskaźnik** jest zmienną, której wartość jest adresem w pamięci operacyjnej. Deklaracja zmiennej wskaźnikowej jest następująca:

```
void *w;
```

- Ze wskaźnikiem możemy stowarzyszyć typ, który nazywamy **typem bazowym wskaźnika**: informujemy w ten sposób kompilator jakiego typu zmiennej powinien się spodziewać pod adresem, który przechowuje wskaźnik. Wówczas deklaracja przybiera postać:

```
typ *w;
```

Operator adresu & oraz operator dostępu *

- Jednoargumentowy operator & zwraca adres zmiennej.
- Jednoargumentowy operator * zwraca wartość zmiennej, która znajduje się pod wskazanym adresem. Operatorem tego używamy w odniesieniu do wskaźników posiadających typ bazowy. Deklaracja `typ *w;` pozwala kompilatorowi stwierdzić ile bajtów pod adresem `w` składa się na wskazywaną zmienną i jak te bajty interpretować.

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, *w;
    w = &a;

    cout << w << endl; // Wypisujemy adres zmiennej a. Zauważ, że
                        // powiedzieliśmy kompilatorowi, że w wskazuje na int.
    cout << *w << endl; // Wypisujemy wartość zmiennej a.
    a++;
    cout << *w << endl; // Wartość zmiennej a uległa zmianie, a zmienna.
                        // Zmienna w cały czas wskazuje na zmienną a.
    (*w)++;             // Modyfikujemy wartość zmiennej wskazywanej przez w.
    cout << a << endl;  // Powyższa instrukcja zmieniła wartość zmiennej a!
    return 0;
}
```

0xbfad1568

5

6

7

Arytmetyka wskaźnikowa

- Na zmiennych wskaźnikowych można wykonywać przypisania, konwersje typów i operacje arytmetyczne, których wynikiem jest inny wskaźnik: **wskaźnik + liczba**, **wskaźnik - liczba**, **wskaźnik++**, **wskaźnik--**.
- Wartość adresu w pamięci, o którą zostanie przesunięty wskaźnik w wyniku dodawania/odejmowania 1, jest równy rozmiarowi typu bazowego wskaźnika (przykładowo, dla **int*** będzie to `sizeof(int)`, czyli np. 4!)

```
#include <iostream>
using namespace std;

#define ROZMIAR 10

int main() {
    int t[ROZMIAR] = { 3, 4, 5, 6 }; // Pozostałe elementy uzupełniane zerami.
    int *w = &(t[0]);               // w przechowuje adres pierwszego elementu tablicy t

    // Zauważ, że jeśli w zawiera adres pierwszego elementu tablicy t, to
    // w+1 jest adresem drugiego elementu, w+2 jest adresem trzeciego, itd.
    for ( int i=0; i < ROZMIAR; i++ )
        cout << *(w++) << ' ';
    cout << endl;
    return 0;
}
```

3,4,5,6,0,0,0,0,0,0,

Wskaźniki a tablice

- W języku C nazwa tablicy jest jednocześnie adresem elementu początkowego (zerowego) tablicy. Nazwa tablicy może więc być traktowana jako wskaźnik.
- Konsekwencja: dla tablicy X , zapis $X + i$ oznacza adres i -tego elementu tablicy (licząc od 0).

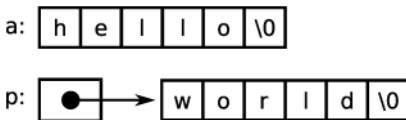
Następujące dwa wyrażenia są równoważne: $X + 3$ i $\&X[3]$. (Ze względu na priorytety operatorów $\&X[3]$ jest równoważne $\&(X[3])$.)

Następujące dwa wyrażenia są równoważne: $*(X + 3)$ i $X[3]$.

Następujące dwa wyrażenia są równoważne: $*X + 3$ i $X[0] + 3$.

- W związku z powyższymi przykładami, należy uważać na **priorytety** operatorów.

```
char a[] = "hello";  
char *p = "world";
```



Wskaźniki a parametry do funkcji

- Następujący program spowoduje wypisanie liczby 0 na ekran.
- Dzieje się tak dlatego, że funkcja `f` otrzymuje na wejściu kopię wskaźnika `&a`. (Zgodnie z ogólną regułą, do funkcji w języku C zawsze przekazywana jest kopia argumentu.)
- Kopia powyższego adresu (czyli zmienna `w`) przechowuje ten sam adres co `&a`, a w konsekwencji instrukcja `*w = 0` poleca umieścić liczbę 0 pod adresem przechowywanym w zmiennej `w`, a więc poleca umieścić liczbę 0 w zmiennej `a`.
- Powyższego mechanizmu można użyć w sytuacji, aby uzyskać efekt modyfikacji argumentów przez funkcję.

```
#include <iostream>
using namespace std;

void f( int *w ) {
    *w = 0;
}

int main() {
    int a = 5;
    f( &a );
    cout << a;
    return 0;
}
```

Manipulacja napisami – przykłady

W języku C nie ma instrukcji pozwalającej na kopiowanie tablic, w szczególności tablic znaków. Aby wykonać kopię napisu, niezbędne jest jego kopiowanie “znak po znaku”.

```
#include <iostream>
using namespace std;

/* Funkcja nie sprawdza czy tablica cel jest wystarczająco pojemna, aby
przechować cały napis (wszystkie znaki aż do wystąpienia znaku '\0')
z tablicy zrodlo */
void kopiuj_napis( const char zrodlo[], char cel[] ) {
    int i;
    for ( i=0; zrodlo[i] != '\0'; i++ )
        cel[i] = zrodlo[i];
    cel[i] = '\0'; // Nie powinniśmy zapominać o '\0' kończącym napis!
}

int main() {
    char c1[] = "Wskaźniki są fajne i bardzo przydatne!";
    char c2[100];

    kopiuj_napis( c1, c2 );
    return 0;
}
```

Manipulacja napisami – przykłady

Deklaracje `typ *w` oraz `typ x[]` są niemalże równoważne:

- `w[i]` oraz `x[i]` są i-tymi obiektami typu `typ` od miejsca wskazywanego przez, odpowiednio, `w` oraz `x`.
- Modyfikacja zawartości `x` nie jest możliwa (tzn. instrukcja `x++` spowoduje błąd kompilacji), natomiast modyfikacja `w` jest możliwa.

Poprzedni program możemy zapisać następująco:

```
#include <iostream>
using namespace std;

/* Funkcja nie sprawdza czy tablica cel jest wystarczająco pojemna, aby
przechować cały napis (wszystkie znaki aż do wystąpienia znaku '\0')
z tablicy zrodlo */
void kopiuj_napis( const char *zrodlo , char *cel ) {
    while ( *zrodlo != '\0' )
        *(cel++) = *(zrodlo++);
    *(cel) = '\0'; // Nie powinniśmy zapominać o '\0' kończącym napis!
}

int main() {
    char c1[] = "Wskaźniki sa fajne i bardzo przydatne!";
    char c2[100];

    kopiuj_napis( c1 , c2 );
    return 0;
}
```

Stałe wskaźniki a wskaźniki na stałą

- Deklaracja:

```
const int *w;
```

powoduje utworzenie wskaźnika `w`, który można używać jedynie do odczytu elementów, na które wskazuje, natomiast nie można użyć `w` do modyfikacji tych elementów. Na przykład, instrukcje `*w=0` lub `w[3]++` spowodują błąd kompilacji.

- Deklaracja:

```
int *const w;
```

powoduje utworzenie stałego wskaźnika. Oznacza to, że można użyć zmiennej `w` do odczytu i modyfikacji wartości wskazywanych przez `w`, lecz nie można zmieniać wartości zmiennej `w` (tzn. nie można zmieniać adresu, który `w` przechowuje). Innymi słowy, `w` wskazuje przez cały czas swojego życia na to samo miejsce w pamięci operacyjnej. Tego typu wskaźniki trzeba inicjalizować (w szczególności, mogą być używane jako parametry funkcji).

- Deklaracja:

```
const int *const w;
```

łączy obie powyższe cechy.

Stałe wskaźniki a wskaźniki na stałą – przykłady

```
int main() {  
    int t[] = { 1, 2 };  
    const int *w;  
    int *const p; // Błąd kompilacji! Ten wskaźnik musi być zainicjalizowany.  
    int *const x=t;  
    const int *const y = x+1;  
    w = t; // Dobrze.  
    w++; // Dobrze.  
    w[0]=0; // Błąd kompilacji! Nie można modyfikować wart. wskazywanych przez w.  
    *w--; // Dobrze, gdyż równoważne: *(w--).  
    (*w)--; // Błąd kompilacji! Próba dekrementacji wart. wskazywanej przez w.  
    x[1] = 3; // Dobrze.  
    x = x+1; // Błąd kompilacji. Nie można zmienić adresu zapisanego w zmiennej x.  
    *t = y[0]; // Dobrze.  
    return 0;  
}
```

Wybrane funkcje z biblioteki string.h

- `size_t strlen( const char *s );`

Funkcja zwraca długość napisu `s`, tzn. liczbę znaków począwszy od miejsca wskazywanego przez `s` aż do pierwszego wystąpienia znaku `'\0'`. `size_t` jest typem całkowitoliczbowym zdefiniowanym w bibliotece standardowej i jest on wystarczająco pojemny, aby przechowywać rozmiary bloków pamięci na danym komputerze. Jeśli znamy górne ograniczenie na długość napisu, to możemy rzutować wynik zwracany przez funkcję do typów całkowitoliczbowych jak `short`, `int`, `long` itp.

- `int strcmp( const char *s1, const char *s2 );`

Funkcja porównuje dwa napisy i zwraca 0 jeśli są identyczne (porównywanie dokonywane jest do napotkania znaku `'\0'`), zwraca liczbę mniejszą niż 0 jeśli `s1` jest mniejszy (alfabetycznie) od `s2`, oraz zwraca liczbę większą niż 0 gdy `s1` jest większy niż `s2`.

- `char *strcpy( char *dest, const char *src );`

Funkcja kopiuje napis wskazywany przez `src` do obszaru pamięci (tablicy) wskazywanego przez `dest`. Programista musi zadbać o to, aby tablica `dest` była wystarczająco pojemna.

Uwaga: zwróć uwagę na kwalifikatory `const`. Ich użycie jest przejawem dobrego stylu programowania, gdyż programista używający tych funkcji wie, że nie mogą one dokonać zmian w napisach `s`, `s1`, `s2` i `src`. **Zadanie:** napisz powyższe funkcje samodzielnie. Także, zapoznaj się z funkcjami `strncmp` i `strncpy` i także napisz je samodzielnie.

Parametry do funkcji – tablice wielowymiarowe

W deklaracji funkcji należy wyspecyfikować:

- typ elementów przekazywanej tablicy,
- wszystkie wymiary tablicy, poza najbardziej zewnętrznym (lewym).

Poniższe deklaracje funkcji są równoważne:

```
void funkcja( int tablica[][3] );
```

```
void funkcja( int (*tablica)[3] );
```

Przykład poprawnego wywołania powyższych funkcji:

```
int t[10][3];  
funkcja( t );
```

Stała NULL

- Czasem zachodzi potrzeba przypisania do zmiennej wskaźnikowej wartości której znaczenie jest takie, że zmienna ta nie wskazuje na żaden obszar pamięci.
- Do powyższego celu służy stała `NULL` (zdefiniowana w `stdlib.h`).
- `NULL` jest stałą typu `void *` a zatem można ją rzutować na każdy typ wskaźnikowy.
- W miejsce `NULL` można użyć `0` lub `(void *)0`, lecz używanie `NULL` jest dobrą praktyką, gdyż sprawia, że kod jest łatwiejszy w czytaniu.

Dynamiczna alokacja pamięci

`stdlib.h` dostarcza dwóch następujących funkcji:

```
void *malloc( size_t size );
```

Funkcja ta przydziela programowi blok pamięci o rozmiarze `size` bajtów i zwraca adres początku tego bloku. Funkcja ta odwołuje się do systemowych mechanizmów przydziału pamięci. Zwraca `NULL` w przypadku niepowodzenia (np. brak wystarczającej ilości wolnej pamięci). Zauważmy, że funkcja zwraca wskaźnik typu `void *`. Od nas zależy jakiego typu elementy chcemy w przydzielonym bloku przechowywać i dokonujemy odpowiedniego rzutowania (patrz kolejny przykład).

```
void free( void *ptr );
```

Funkcja zwalnia pamięć przydzieloną wcześniej przez `malloc` (lub inną funkcję przydziału pamięci dostępną w C). Jeśli przekazujemy wskaźnik do bloku pamięci zwolnionej wcześniej, zachowanie funkcji `free` jest niezdefiniowane (taka sytuacja jest błędem logicznym programisty). Jeśli `free` otrzyma na wejściu wskaźnik `NULL`, to żadna akcja nie jest wykonywana.

Dynamiczna alokacja pamięci – typowy schemat postępowania

- Założmy, że potrzebujemy zaalokować blok pamięci, w którym będziemy przechowywali `n` obiektów typu `typ`.
- Deklarujemy najpierw wskaźnik, w którym będziemy przechowywali adres początku tego bloku: `typ *blok = NULL;`
- Inicjalizacja wskaźnika wartością `NULL` nie jest konieczna.
- Używamy funkcji `malloc` aby przydzielić pamięć:
`blok = (typ *) malloc( n*sizeof(typ) );`
- Zauważmy, że zamiast szacować ile bajtów pamięci operacyjnej zajmuje obiekt typu `typ`, pozostawiamy tą pracę operatorowi `sizeof` – w ten sposób nigdy się nie pomylimy i przydzielony blok ma właściwy rozmiar. Zwróć uwagę na rzutowanie z `void *` (typ zwracany przez `malloc`) do typu naszego wskaźnika.
- Warto sprawdzić czy blok pamięci został faktycznie przydzielony:
`if ( blok == NULL ) { tutaj obsługa sytuacji braku pamięci }`
- Gdy pamięć nie jest już więcej potrzebna, wywołujemy:
`free( blok );`

Powyższy mechanizm jest zilustrowany na kolejnym slajdzie, gdzie pokazany jest program wczytujący od użytkownika ciąg liczb i wypisujący te liczby w odwrotnej kolejności. Z góry nie wiemy ile liczb użytkownik chce wpisać (to użytkownik podaje na początku programu), więc musimy przydzielać pamięć w trakcie działania programu.

Dynamiczna alokacja pamięci – przykład

```
#include <iostream>
#include <stdlib.h>
using namespace std;

void wczytaj( int t[], int n ) {
    for ( int i=0; i < n; i++ ) {
        cout << "Podaj " << i << "-ty element: ";
        cin >> t[i];
    }
}

void wypisz_odwrotnie( const int *t, int n ) {
    for ( int i=n-1; i >= 0; i-- )
        cout << "t[" << i << "] = " << t[i] << endl;
}

int main() {
    int n, *p = NULL;
    cin >> n; // Wczytujemy liczbę elementów od użytkownika.
    p = (int *) malloc( n*sizeof(int) );
    if ( p == NULL ) { // Koniecznie sprawdzamy, czy system operacyjny
        cout << "Brak pamięci!\n"; // faktycznie przydzielił żadaną
        return 1; // pamięć. Jeśli nie – kończymy program.
    }
    wczytaj( p, n );
    wypisz_odwrotnie( p, n );
    free( p ); // Pamiętamy o zwolnieniu pamięci – to dobry nawyk.
    return 0;
}
```

```
5
Podaj 0-ty element: 1
Podaj 1-ty element: 3
Podaj 2-ty element: 5
Podaj 3-ty element: 7
Podaj 4-ty element: 9
t[4] = 9
t[3] = 7
t[2] = 5
t[1] = 3
t[0] = 1
```

```
10000000000
Brak pamięci!
```

Parametry linii poleceń

- Poniższy program ilustruje pewien mechanizm pozwalający na komunikację programisty z "otoczeniem".
- Mianowicie, zadeklarowanie funkcji `main` z takimi parametrami, pozwala na dostęp do parametrów (`argv`) oraz ich liczby (`argc`) podawanych z linii poleceń przy uruchomieniu programu.
- `argv[i]` jest napisem będącym i-tym spośród tych parametrów (numerujemy jak zawsze od zera). Poprawne wartości indeksu `i` są z zakresu od `0` do `argc-1`.
- `argv[0]` jest zawsze nazwą pliku wykonywalnego z nazwą programu.

```
deren@linux-nb6o:~> ./w07p08 -o -a -c ała ma kota
7
0-ty argument to: ./w07p08
1-ty argument to: -o
2-ty argument to: -a
3-ty argument to: -c
4-ty argument to: ała
5-ty argument to: ma
6-ty argument to: kota
```

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    cout << argc << endl;
    for ( int i=0; i < argc; i++ )
        cout << i << "-ty argument to: " << argv[i] << endl;
    return 0;
}
```

Podstawy Programowania

Wykład nr 8: Rekurencja

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Rekurencja – ogólna idea

Rekurencja – sposób implementacji algorytmu, polegający na zagnieżdżonym wywoływaniu funkcji z poziomu jej samej, najczęściej w celu rozwiązania „podproblemu” większego zadania obliczeniowego:

- rekurencja pozwala na łatwiejsze rozwiązywanie zadań zdefiniowanych indukcyjnym przepisem,
- w językach programowania, innych niż języki funkcyjne, procedura zostanie uruchomiona każdorazowo, nawet w przypadku wywołania z tym samym zestawem parametrów, co przy wcześniejszym wywołaniu,
- zmienne lokalne oraz argumenty wywołania funkcji są każdorazowo odkładane na stosie,
- rekurencja wiąże się ze zwiększonym narzutem pamięciowym (każde wywołanie funkcji powoduje odłożenie na stosie wskaźnika instrukcji oraz zawartości rejestrów w momencie wywołania) oraz narzutem czasowym.

Uwaga: każdą procedurę rekurencyjną można zastąpić przez równoważną procedurę iteracyjną (używając tylko pętli i pomocniczej struktury danych – stosu), nie zmieniając w sposób istotny czasu działania programu i ilości użytej pamięci.

Przykład 1: liczby Fibonacciego

- n -tą liczbę Fibonacciego, gdzie $n > 0$ jest liczbą całkowitą, obliczamy wg wzoru:

$$f_n = \begin{cases} 1, & \text{gdy } n = 1 \text{ lub } n = 2 \\ f_{n-2} + f_{n-1}, & \text{gdy } n > 2 \end{cases}$$

- Deklaracja funkcji obliczającej liczbę Fibonacciego o numerze n mogłaby wyglądać następująco: `int fib( int n );`. Jest ona oczywiście niezależna od sposobu implementacji (tzn. na przykład od tego czy funkcja będzie rekurencyjna czy też nie).

```
#include <iostream>
using namespace std;

/* Funkcja zwracająca n-tą liczbę Fibonacciego. Wywołanie funkcji
   z n <= 0 jest niepoprawne i wówczas funkcja zwraca 1 */
int fib( int n ) {
    if ( n <= 2 )
        return 1;
    else
        return fib(n-2) + fib(n-1);
}

int main() {
    for ( int i=1; i <= 10; i++ )
        cout << " fib(" << i << ") = " << fib(i) << endl;
    return 0;
}
```

```
fib(1) = 1
fib(2) = 1
fib(3) = 2
fib(4) = 3
fib(5) = 5
fib(6) = 8
fib(7) = 13
fib(8) = 21
fib(9) = 34
fib(10) = 55
```

Przykład 2: zliczanie znaków w napisie

Napiszmy funkcję, która zlicza liczbę wystąpień zadanego znaku w podanym na wejściu napisie. Taka funkcja oczywiście przyjmie jako argument napis (typu `char *`). Skoro funkcja nie ma potrzeby modyfikować tego napisu (a nawet nie powinna tego robić!), użyjemy kwalifikatora `const`. Drugim argumentem będzie zliczany znak (typu `char`).

```
#include <iostream>
using namespace std;

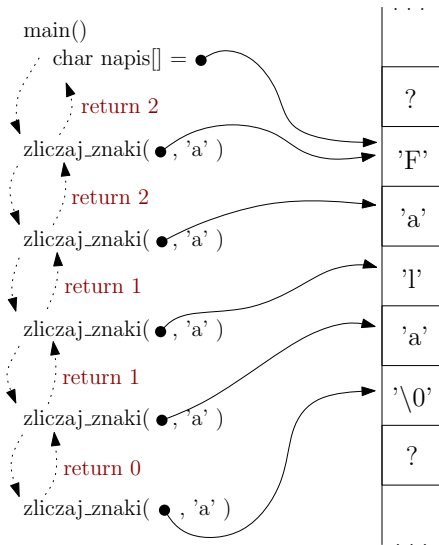
/* Funkcja zwraca liczbę całkowitą równą liczbie wystąpień znaku podanego
   jako drugi argument w napisie s */
int zliczaj_znaki( const char *s, char znak ) {
    if ( *s == '\0' ) // jeśli napis jest pusty, to zwracamy 0
        return 0;
    else {
        int licz = zliczaj_znaki( s+1, znak );
        if ( *s == znak )
            licz++;
        return licz;
    }
}

int main() { // Dołączamy prosty program testowy.
    char napis[] = { "Fala" };
    cout << zliczaj_znaki( napis, 'a' ) << endl;
    return 0;
}
```

Przykład 2: zliczanie znaków w napisie c.d.

Rysunek ilustruje kolejne wywołania funkcji `zliczaj_znaki`. Zauważ, że:

- W pamięci operacyjnej znajduje się tylko jedna kopia napisu "Fala".
- Poszczególne instancje funkcji `zliczaj_znaki` otrzymują kopie wskaźnika na odpowiednie miejsce w pamięci operacyjnej.
- Każda instancja funkcji `zliczaj_znaki`, za wyjątkiem ostatniej, posiada swoją własną kopię zmiennej `licz`.
- W związku z powyższym, przy czwartym wywołaniu funkcji `zliczaj_znaki` tworzona jest (na stosie) czwarta kopia zmiennej `licz`.



Przykład 3: wyszukiwanie największego elementu w tablicy

```
#include <iostream>
using namespace std;

int najwiekszy( int t[], int rozmiar ) {
    if ( rozmiar == 1 )
        return 0;
    else {
        int a = najwiekszy( t, rozmiar-1 );
        if ( t[a]>t[rozmiar-1] )
            return a;
        else
            return rozmiar-1;
    }
}
```

Funkcja `int najwiekszy( int t[], int rozmiar )` dzięki wywołaniu siebie samej:

`a = najwiekszy( t, rozmiar-1 );`

dowiaduje się jaki jest indeks `a` taki, że `t[a]` jest największą liczbą spośród `t[0]`, `t[1]`, ..., `t[rozmiar-2]`. Innymi słowy, znajdujemy indeks największego elementu tablicy `t` spośród jej pierwszych `rozmiar-1` elementów.

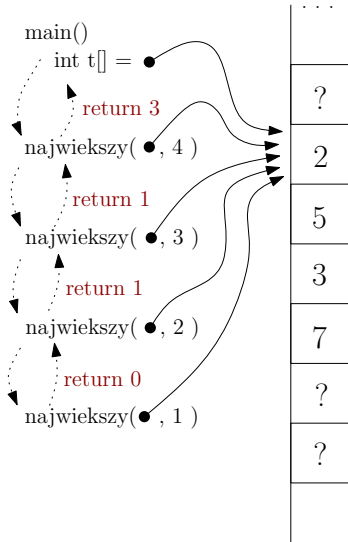
Oznacza to, że indeks największego elementu spośród pierwszych `rozmiar` elementów to `a` lub `rozmiar-1`.

Przykład 3: wyszukiwanie największego elementu w tablicy c.d.

```
#include <iostream>
using namespace std;

int najwiekszy( int t[], int rozmiar ) {
    if ( rozmiar == 1 )
        return 0;
    else {
        int a = najwiekszy( t, rozmiar-1 );
        if ( t[a]>t[rozmiar-1] )
            return a;
        else
            return rozmiar-1;
    }
}
```

- Wywołanie **najwiekszy** dla tablicy {2,5,3,7} (**rozmiar** jest równy 4) jest pokazane na rysunku.
- Zauważmy, że dla tablicy rozmiaru **n** nastąpi dokładnie **n** wywołań funkcji **najwiekszy**.



Przykład 3: porządkowanie tablicy

```
#include <iostream>
using namespace std;

int najwiekszy( int t[], int rozmiar ) { // Funkcja zwraca indeks
    if ( rozmiar == 1 ) // w tablicy t, pod którym
        return 0; // znajduje się jej
    else { // największy element.
        int a = najwiekszy( t, rozmiar-1 );
        return ( t[a]>t[rozmiar-1] ? a : rozmiar-1 );
    }
}

void zamien(int t[], int indeks1, int indeks2 ) {
    int pomocnicza = t[indeks1]; // Funkcja dokonuje zamiany elementów
    t[indeks1] = t[indeks2]; // pod indeksami indeks1 i indeks2
    t[indeks2] = pomocnicza; // w tablicy t
}

void porzadkuj_tablice( int t[], int n ) { // Funkcja porządkuje
    if ( n > 1 ) { // niemalejąco liczby
        zamien( t, n-1, najwiekszy( t, n ) ); // w talicy t o rozmiarze n
        porzadkuj_tablice( t, n-1 );
    }
}

int main() { // Dołączamy prosty program testowy.
    int t[] = { 0, 41, 0, 4, 0, 9, 0, 4, 5, 3, 4, 5, 6, 0, -3, -3, -1, 20 };

    porzadkuj_tablice( t, (sizeof t)/(sizeof t[0]) );
    for ( int i=0; i < (sizeof t)/(sizeof t[0]); i++ )
        cout << t[i] << endl;
    return 0;
}
```

Przykład 3: porządkowanie tablicy – komentarz do kodu

- W powyższej implementacji, funkcje `najwiekszy` oraz `porzadkuj_tablice` są rekurencyjne.
- Funkcja `porzadkuj_tablice` najpierw sprawdza czy $n > 1$. Działanie funkcji jest kontynuowane tylko wówczas, gdy warunek ten jest prawdziwy, gdyż w przeciwnym razie zadaniem takiego wywołania funkcji jest uporządkowanie jedno-elementowej tablicy – w takim wypadku niczego nie trzeba robić.
- Jeśli $n > 1$, to funkcja `porzadkuj_tablice` dokonuje zamiany elementów, tak aby największy znalazł się na jej końcu, czyli pod indeksem $n-1$. Wywołanie `porzadkuj_tablice( t, n-1 )` porządkuje pierwsze $n-1$ elementów tablicy `t`. Zauważ, że gdy to zostanie wykonane, tablica jest już uporządkowana!

Podstawy Programowania

Wykład nr 9: Wejście/wyjście

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Wejście i wyjście w C

- `#include <stdio.h>` pozwala korzystać z funkcji standardowego wejścia i wyjścia w języku C
- funkcje wejścia/wyjścia zwracają wartości, które pozwalają na wykrycie ewentualnych sytuacji błędnych/szczególnych
- sytuacje takie to np.:
 - brak danych przy próbie czytania z wejścia
 - zły format danych wejściowych
 - błąd przy wypisywaniu danych na wyjście

Standardowe wyjście

```
int putchar( int c );
```

Funkcja wypisuje znak o kodzie ASCII `c` na standardowe wyjście i zwraca kod ASCII wypisanego znaku jako `unsigned char` rzutowany na `int`, oraz zwraca stałą `EOF` w przypadku błędu.

```
int printf( const char *format, ... );
```

- Funkcja `printf` wypisuje na standardowe wyjście napis `format`, w którym wartości sekwencji kontrolnych, rozpoczynających się od znaku `%` są zastępowane wartościami zmiennych podanych jako dodatkowe argumenty w miejsce `....`
- `printf` zwraca liczbę znaków, które zostały wypisane.

Sekwencje kontrolne

Sekwencje kontrolne:

Sekw.	Typ	Co wypisuje?
%d	int	wartość dziesiętną
%c	char	znak o zadanym kodzie ASCII
%s	char *	napis
%p	void *	adres
%f	float	wartość liczby zmiennoprzec.
%e	float	j.w., lecz notacja wykładnicza

Modyfikatory:

Mod.	Działanie.
l	"long"
0	uzup. puste pola zerami
liczba	określa szerokość pola
.liczba	liczba miejsc po przec.

```
#include <stdio.h>
#include <math.h>
```

```
int main() {
    double pi = acos(-1.0), potega;
    int i;
    char c = 'a';
    long l = 10;
    long long ll = 20;
```

```
    for ( i=0, potega=pi; i < 5; i++, potega *= pi )
        printf( "%.4f do potęgi %2d = %.6le\n", (float)pi, i+1, potega );
```

```
    printf( "Kod ASCII litery %c wynosi %d\n", c, (int)c );
    printf( "Liczba typu long: %ld,\n... oraz typu long long: %lld\n", l, ll );
    printf( "Adres zmiennej l to %p\n", (void *)&l );
    return 0;
}
```

```
3.1416 do potęgi 1 = 3.141593e+00
3.1416 do potęgi 2 = 9.869604e+00
3.1416 do potęgi 3 = 3.100628e+01
3.1416 do potęgi 4 = 9.740909e+01
3.1416 do potęgi 5 = 3.060197e+02
Kod ASCII litery a wynosi 97
Liczba typu long: 10,
... oraz typu long long: 20
Adres zmiennej l to 0xbfd1134
```

Standardowe wejście

```
int getchar( void );
```

Funkcja pobiera jeden znak ze standardowego wejścia i zwraca ten znak jako `unsigned char` rzutowany na `int`, oraz zwraca stałą `EOF` w przypadku błędu lub końca pliku (końca wejścia)

```
int scanf( const char *format, ... );
```

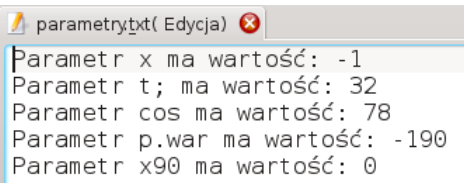
- Funkcja `scanf`, zgodnie z formatem podanym w napisie `format`, dopasowuje ciąg znaków ze standardowego wejścia, modyfikując wartości podanych zmiennych.
- `scanf` zwraca liczbę dopasowanych parametrów.
- Uwaga: parametry `...` to adresy zmiennych.

scanf – przykład

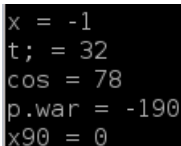
Mamy dany plik o nazwie `parametry.txt`, którego każda linia ma postać: "Parametr ... ma wartość: ...", gdzie w miejscu pierwszego "..." mamy dowolne słowo (bez białych znaków) natomiast w miejscu drugiego "..." mamy liczbę całkowitą. Interesuje nas odcodowanie nazw wszystkich parametrów oraz ich wartości. Plik ten zostanie przekierowany na standardowe wejście, tzn program o nazwie `prog.exe` uruchomimy następująco: `prog.exe <parametry.txt`.

```
#include <stdio.h>
```

```
int main() {  
    char parametr[128];  
    int wartosc;  
  
    while ( scanf( "Parametr %s ma wartość: %i\n", parametr, &wartosc ) > 0 )  
        printf( "%s = %d\n", parametr, wartosc );  
    return 0;  
}
```



```
parametry.txt( Edycja) x  
Parametr x ma wartość: -1  
Parametr t; ma wartość: 32  
Parametr cos ma wartość: 78  
Parametr p.war ma wartość: -190  
Parametr x90 ma wartość: 0
```



```
x = -1  
t; = 32  
cos = 78  
p.war = -190  
x90 = 0
```

Otwieranie i zamykanie plików

```
FILE *fopen( const char *sciezka, const char *tryb );
```

- Typ **FILE *** przechowuje wskaźnik na specjalny typ opisujący plik.
- Funkcja zwróci **NULL** jeśli otwarcie pliku nie powiodło się.
- Napis **tryb** zaczyna się od jednej z następujących sekwencji kontrolnych:
 - **r** – otwarcie pliku do odczytu.
 - **r+** – otwarcie pliku do odczytu i zapisu.
 - **w** – utwórz pusty plik do zapisu; jeśli plik istnieje, to jest nadpisany pustym.
 - **w+** – otwarcie pliku do zapisu i odczytu. Jeśli plik istnieje, to jest nadpisany pustym. We wszystkich dotychczasowych przypadkach, wskaźnik odczytu/zapisu do pliku jest inicjalnie ustawiany na jego początek.
 - **a** – otwarcie pliku w celu dopisywania na jego końcu. Plik jest tworzony jeśli nie istnieje.
 - **a+** – otwarcie pliku do odczytu i dopisywania na jego końcu. Wskaźnik odczytu jest ustawiany na początek pliku, lecz zapisywanie do pliku odbywa się na jego końcu.
- Ustawienie wskaźnika odczytu/zapisu pliku można dokonać funkcją **fseek**.
- Plik zamykamy używając funkcji **int fclose(FILE *);**. (Zwraca **EOF** w przypadku błędu.)

Zapisywanie/odczyt do/z pliku (tryb tekstowy)

Do zapisywania (formatowanego/tekstowego) do pliku służą m.in. funkcje:

```
int fprintf( FILE *f, const char *format, ... );  
int fputc( int c, FILE *f);  
int fputs( const char *napis, FILE *f);
```

Do odczytywania (formatowanego/tekstowego) z pliku służą m.in. funkcje:

```
int fscanf( FILE *f, const char *format, ... );  
int fgetc( FILE *f);  
int fgets( const char *napis, int rozmiar, FILE *f);
```

Przy używaniu powyższych funkcji obowiązują analogiczne zasady jak przy ich odpowiednikach działających na standardowym wejściu/wyjściu, z wyjątkiem dodatkowego parametru wskazującego na plik.

Zapisywanie/odczyt do/z pliku (tryb tekstowy) – przykład

```
#include <stdio.h>
#define LPOTEG 20
/* program wpisuje kolejne potęgi liczby 2 do pliku potegi.txt.
   Kolejne liczby są oddzielone w pliku znakiem nowej linii */
int main() {
    FILE *plik;
    unsigned long long pot2 = 1;

    plik = fopen( "potegi.txt", "w" );
    if ( plik == NULL ) {
        printf( "Otwarcie pliku nie powiodło się\n" );
        return 1;
    }

    for ( int i=0; i < LPOTEG; i++ )
        fprintf( plik, "%lld\n", pot2<<i );

    fclose( plik );
    return 0;
}
```

```
1
2
4
8
16
32
64
128
256
512
1024
2048
4096
8192
16384
32768
65536
131072
262144
524288
```

Zapisywanie/odczyt do/z pliku (tryb binarny)

Do zapisywania (binarnego) do pliku służy funkcja:

```
size_t fwrite( const void *w, size_t rozmiar, size_t l_el, FILE *f);
```

 Do

odczytywania (binarnego) z pliku służy funkcja:

```
size_t fread( void *w, size_t rozmiar, size_t l_el, FILE *f);
```

- Funkcje zapisują/odczytują `l_el` elementów, każdy rozmiaru `rozmiar`, znajdujących się w pamięci pod adresem `w`.
- Obie funkcje zwracają liczbę odczytanych lub zapisanych elementów (spośród `l_el` wskazanych do zapisu/odczytu). Aby otrzymać łączną liczbę zapisanych/odczytanych bajtów, należy zwróconą wartość wymnożyć przez wartość parametru `rozmiar`.

Przykład: zapis binarny tablicy liczb do pliku

```
#include <stdio.h>
#define MAX 20
/* program wpisuje kolejne potęgi liczby 2 do pliku potegi.txt.
   Kolejne liczby są oddzielone w pliku znakiem nowej linii */
int main() {
    FILE *plik;
    int tab[MAX], i;

    /* wypełniamy tablicę kolejnymi liczbami naturalnymi */
    for ( i=0; i < MAX; i++ )
        tab[i] = i;

    plik = fopen( "liczby.bin", "w" );
    if ( plik == NULL ) {
        printf( "Otwarcie pliku nie powiodło się\n" );
        return 1;
    }

    fwrite( (void *)tab, sizeof tab[0], MAX, plik );
    fclose( plik );
    return 0;
}
```

- Plik `liczby.bin` będzie miał dokładnie $\text{MAX} * (\text{sizeof tab}[0])$ bajtów, bez względu na zawartość tablicy `tab`.
- *i*-ta liczba z tablicy `tab` (traktując liczbę pod indeksem 0 jako pierwszą) jest zapisana na kolejnych `sizeof(int)` bajtach, począwszy od bajtu numer $(i-1) * \text{sizeof(int)}$.

Podstawy Programowania

Wykład nr 10: Typy danych użytkownika. Dynamiczne struktury danych.

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Typ wyliczeniowy enum

- Przypuśćmy, że chcemy w programie zadeklarować zmienne, które będą przechowywać wartości odpowiadające kierunkom.
- Możliwe podejście jest następujące. Deklarujemy stałe, które będą reprezentować poszczególne kierunki:

```
#define LEWO 0  
#define PRAWO 1  
#define GORA 2  
#define DOL 3
```

- Następnie, używamy zmiennych typu `int` do przechowywania wartości.
- Rozwiązaniem bardziej polecanym jest użycie typu `enum`

Typ wyliczeniowy enum

Typy wyliczeniowy jest „okrojona” formą zmiennych całkowitych (typu int), w których zamiast wartości liczbowych używamy własnych etykiet.

```
enum kierunek_t { /* kierunek_t jest nazwą typu */  
    LEWO,        /* pierwszy element ma wartość 0 */  
    PRAWO,       /* drugi element ma wartość 1 */  
    GORA,        /* itd. */  
    DOL  
} a, b; /* a i b to zmienne typu enum kierunek_t */
```

- Przyrostek “_t” jest jedynie konwencją: widać wyraźnie, że identyfikator ten jest nazwą typu, co odróżnia go od identyfikatorów stałych, zmiennych, czy funkcji.
- Późniejsza deklaracja zmiennej: `enum kierunek_t a = LEWO;`
- Przypisania: `a = DOL;`
- **Uwaga:** język C++ ma ścisłą kontrolę typu; w szczególności nie można do zmiennej typu `kierunek_t` przypisywać wartości typu `int` (na przykład `a=0` spowoduje błąd kompilacji). Takiej kontroli nie ma w C.

Typ enum – uwagi

- Stałym typu wyliczeniowego można przypisać własne wartości. We wcześniejszym przykładzie, w deklaracji typu `kierunek_t` możemy zastąpić `GORA` przez `GORA=0`. Oczywiście możemy użyć dowolnej innej liczby typu `int` zamiast 0.
- Wartości liczbowe są przypisywane domyślnie do stałych, którym wartości nie przypisujemy: wartość identyfikatora jest o jeden większa od wartości identyfikatora poprzedniego.
- W wyniku inicjalizacji `GORA=0`, `DOL` przyjmie wartość 1. **Uwaga:** w tej sytuacji `PRAWO == DOL` będzie prawdą!
- Na stałych i zmiennych wyliczeniowych (dotyczy C++) nie można wykonywać operacji arytmetycznych. Na przykład wyrażenia `DOL+1`, `a++` lub `a+3` są niepoprawne, gdzie `a` jest zmienną typu `kierunek_t`.

Typ agregacyjny (struct)

Struktury pozwalają na zapisywanie rekordów, to jest zapamiętanie **wszystkich** spośród wymienionych w definicji typu pól.

```
struct student_t { /* student_t jest nazwą typu */
    char imie[15];
    char nazwisko[25];
    unsigned int nr_albumu;
    char rok_studiow;
} stud1, stud2; /* stud1 i stud2 to zmienne typu struct student_t */
```

- Identyfikator `student_t` po słowie kluczowym `struct` jest opcjonalny. W przypadku jego braku mamy do czynienia z typem anonimowym.
- Identyfikatory `stud1` oraz `stud2` są również opcjonalne: w przypadku ich braku mamy do czynienia jedynie z deklaracją typu.
- Późniejsza deklaracja zmiennych: `struct student_t ktos;`

Rozróżnialność typów struktur

- Każda deklaracja struktury jest definicją nowego typu, nawet jeśli deklaracja ta jest identyczna do deklarowanej wcześniej struktury. Na przykład:

```
struct a_t { int x; };  
struct b_t { int x; };
```

powoduje utworzenie dwóch różnych typów o nazwach `a_t` i `b_t`.

- W przypadku deklaracji:

```
struct a_t a1, a2;  
struct b_t b;
```

zmienne `a1` oraz `a2` są tego samego typu, natomiast `a1` oraz `b` są innych typów.

- Powyższe oznacza w szczególności, że przypisanie `b = a1`; spowoduje błąd kompilacji.

Odwołania do składowych struktury

- Metoda podstawowa:

```
nazwa_zmiennej_strukturalnej.nazwa_składowej
```

- Metody pośrednie. Jeśli dysponujemy adresem (wskaźnikiem) zmiennej strukturalnej, to do poszczególnych składowych możemy się odwołać używając jednoargumentowego operatora *:

```
(*wskaznik).nazwa_skladnika
```

lub używając operatora ->:

```
wskaznik->nazwa_skladnika
```

Zagnieżdżanie struktur

- Polem składowym struktury może być inna struktura.
- W takich przypadkach struktury zagnieżdżone są zwykle anonimowe.
- Przykład:

```
struct obiekt_t {  
    char nazwa[15];  
    struct {  
        int x;  
        int y;  
    } wspolzedne;  
} o1, o2;
```

Wówczas możliwe odwołania do składowych:

`o1.wspolzedne.x`

`o1.wspolzedne = o2.wspolzedne;`

W drugim przypadku występuje zgodność typów i operator `=` przypisuje wartości składowych struktury `o2.wspolzedne` do odpowiednich składowych struktury `o1.wspolzedne`.

Inicjalizacja zmiennych strukturalnych

Zmienną strukturalną możemy inicjalizować na różne sposoby:

- Podając w nawiasach klamrowych wartości poszczególnych składowych. Kolejność podania wartości musi być zgodna z kolejnością deklaracji poszczególnych składowych.
- Podając składowe w sposób jawny. Kolejność inicjalizacji jest dowolna. W tej metodzie oraz w metodzie powyższej, składowe którym nie przypisano wartości są inicjalizowane wartością 0. **Uwaga:** metoda dostępna tylko w C.
- Dokonując przypisania wartości innej zmiennej strukturalnej (tego samego typu).

```
struct punkt_t { int x,y; };

int main() {
    struct punkt_t p1 = {2,3}; // metoda 1: podanie składowych
    struct punkt_t p2 = {.y=4,.x=6}; // metoda 2: jawne odwołanie do składowych
    struct punkt_t p3 = p1; // metoda 3: kopiowanie zawartości innej struktury

    struct punkt_t p4 = {2}; // p4.x=2, p4.y=0
    struct punkt_t p5 = {.y=3}; // p5.x=0, p5.y=3

    return 0;
}
```

Tablice struktur

- Elementami tablicy mogą być struktury. Przykładowa deklaracja takiej tablicy jest następująca:

```
struct typ_t t[N];
```

- Wówczas `t[i]` jest typu `struct typ_t`, gdzie `i` jest indeksem z zakresu od 0 do `N-1`.
- Aby odwołać się do składowej struktury numer `i` w tablicy `t`, piszemy:

```
t[i].składowa
```

- Powyższe jest równoważne `(t[i]).składowa` ze względu na priorytety operatorów.

Uwaga: przekazywanie struktur do funkcji odbywa się, tak jak w przypadku każdej innej zmiennej, przez "wartość", tzn. kopia struktury jest przekazywana jako argument. Jeśli polem struktury jest tablica, to następuje kopiowanie tablicy.

Unie

- Unie deklaruje się analogicznie, jak struktury, przy czym używa się słowa kluczowego `union` zamiast `struct`.
- W przeciwieństwie do struktury, unia nie przechowuje w pamięci wszystkich swoich pól, ale tylko to pole, które zostało zapisane jako **ostatnie**.
- Odwołania do innych pól unii (tzn. wszystkich z wyjątkiem ostatnio zapisanego) mogą prowadzić do nieokreślonego działania.
- Rozmiar zajmowany w pamięci przez unię odpowiada rozmiarowi jej największego pola.

Unie – przykład

```
#include <iostream>
using namespace std;

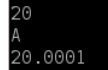
enum typy_t { FLOAT, CHAR };
struct zmienny_typ_t { // W zmiennej typu zmienny_typ_t będziemy zawsze
    enum typy_t typ; // przechowywać jedną wartość, wartość typu float lub char.
    union { // O tym, jakiego typu wartość jest przechowywana w danym
        float liczba; // momencie będzie nas informować zmienna wyliczeniowa typ.
        char znak; // Np, jeśli chcemy przechować zmienną typu float, to przypisujemy
    } wartosc; // ją do pola wartosc.liczba i ustawiamy pole typ na FLOAT.
};

struct zmienny_typ_t przypisz_float( float d ) {
    struct zmienny_typ_t p;
    p.typ = FLOAT; p.wartosc.liczba = d;
    return p;
}

struct zmienny_typ_t przypisz_znak( char c ) {
    struct zmienny_typ_t p;
    p.typ = CHAR; p.wartosc.znak = c;
    return p;
}

void wypisz_wartosc( struct zmienny_typ_t a ) { // Wiemy, że tylko jedno z pól a.wartosc.
    liczba
    switch ( a.typ ) { // lub a.wartosc.znak jest poprawne. Które? O tym mówi pole a.typ.
        case FLOAT: cout << a.wartosc.liczba << endl; break;
        case CHAR: cout << a.wartosc.znak << endl; break;
    }
}

int main() {
    struct zmienny_typ_t a;
    a = przypisz_float( 20 ); wypisz_wartosc( a ); // Wypiszemy 20.
    a = przypisz_znak( 'A' ); wypisz_wartosc( a ); // Wypiszemy znak A.
    cout << a.wartosc.liczba << endl; // to pole nie jest zainicjalizowane!
    return 0;
}
```



20
A
20.0001

Definicje własnych typów: typedef

Słowo kluczowe **typedef** pozwala na definicje własnych typów. Składnia jest następująca:

typedef typ identyfikator;

Przykłady:

```
#include<iostream>
using namespace std;

typedef unsigned char bit_t;

typedef struct {
    int x, y;
} punkt_t;

typedef struct {
    char c;
    punkt_t wsp;
} znak_t;

int main() {
    bit_t b = (bit_t)0;
    znak_t t[10];

    cout << sizeof( char ) << endl;
    cout << sizeof( int ) << endl;
    cout << sizeof( punkt_t ) << endl;
    cout << sizeof( znak_t ) << endl;
    return 0;
}
```

1
4
8
12

- Rozmiar struktury jest nie mniejszy niż suma rozmiarów jej składowych.
- Rozmiar struktury może przekraczać sumę rozmiarów jej składowych.
- Powyższy efekt jest związany z tzw. efektem wyrównywania położenia poszczególnych składowych do granicy słowa maszynowego (zwykle 2,4 lub 8 bajtów).
- Powyższy efekt jest podyktowany kwestiami związanymi z wydajnością (szybszy dostęp do pól składowych) przez procesor.

Unie – zmodyfikowany przykład

```
#include <iostream>
using namespace std;

typedef enum { FLOAT, CHAR } typy_t;
typedef struct {          // W zmiennej typu zmienny_typ_t będziemy zawsze
    typy_t typ;           // przechowywać jedną wartość, wartość typu float lub char.
    union {               // O tym, jakiego typu wartość jest przechowywana w danym
        float liczba;     // momencie będzie nas informować zmienna wyliczeniowa typ.
        char znak;        // Np, jeśli chcemy przechować zmienną typu float, to przypisujemy
    } wartosc;           // ją do pola wartosc.liczba i ustawiamy pole typ na FLOAT.
} zmienny_typ_t;

zmienny_typ_t przypisz_float( float d ) {
    zmienny_typ_t p;
    p.typ = FLOAT; p.wartosc.liczba = d;
    return p;
}

zmienny_typ_t przypisz_znak( char c ) {
    zmienny_typ_t p;
    p.typ = CHAR; p.wartosc.znak = c;
    return p;
}

void wypisz_wartosc( zmienny_typ_t a ) { // Wiemy, że tylko jedno z pól a.wartosc.liczba
    switch ( a.typ ) { // lub a.wartosc.znak jest poprawne. Które? O tym mówi pole a.typ.
        case FLOAT: cout << a.wartosc.liczba << endl; break;
        case CHAR: cout << a.wartosc.znak << endl; break;
    }
}

int main() {
    zmienny_typ_t a;
    a = przypisz_float( 20 ); wypisz_wartosc( a ); // Wypiszemy 20.
    a = przypisz_znak( 'A' ); wypisz_wartosc( a ); // Wypiszemy znak A.
    cout << a.wartosc.liczba << endl; // to pole nie jest zainicjalizowane!
    return 0;
}
```

Dynamiczne struktury

Przykłady dynamicznych struktur danych:

- tablica o dynamicznie dostosowywanym rozmiarze
- wektor
- lista jednokierunkowa
- lista dwukierunkowa

Przykłady zastosowań dynamicznych struktur danych:

- stos
- kolejka

Tablica o dynamicznie dostosowywanym rozmiarze

- Tablica w języku C charakteryzuje się rozmiarem zadany przy inicjalizacji i nie może być w łatwy sposób rozszerzona.
- W przypadku, gdy chcemy zwiększyć rozmiar bloku pamięci złożonego z a komórek, poczynając od adresu wskazywanego przez wskaźnik `ptr`, należy dokonać realokacji:
 - Alokujemy pewną liczbę A komórek pamięci ($A > a$) pod pewnym adresem `newPtr`. Po tym kroku mamy zaalokowane obydwa bloki pamięci.
 - Przepisujemy zawartość a kolejnych komórek pamięci spod adresu `ptr` pod adres `newPtr`.
 - Zwalniamy blok pamięci pod adresem `ptr`.
 - Podmieniamy wskaźnik `ptr`, by wskazywał na nowy adres bloku (instrukcja: `ptr = newPtr`).
- Realokacja jest operacją kosztowną czasowo (przepisanie danych), dlatego też należy dokonywać tego typu operacji możliwie rzadko.
- Typowym podejściem pozwalającym na ograniczenie liczby realokacji jest np. podwajanie rozmiaru nowo alokowanego bloku względem rozmiaru bloku poprzedniego.

Wektor

- **Wektorem** nazywamy dynamiczną strukturę danych, złożoną z pary liczb całkowitych a, n oraz bloku a komórek pamięci zaalokowanych pod adresem wskazywanym przez wskaźnik `ptr`.
- Wektor zachowuje się tak, jak tablica n -elementowa: w zakresie indeksów $i \in \{0, \dots, n-1\}$, wartość `ptr[i]` można odczytywać bądź zapisywać, jak dla tablicy.
- Operacja `push_back(x)` powoduje wstawienie wartości x na koniec wektora, tzn. kolejno:
 - zwiększenie wartości n o 1,
 - realokację (zwiększenie) bloku pamięci `ptr` w przypadku, gdy $n > a$,
 - dodanie na koniec bloku elementu o wartości x (`ptr[n-1] = x`).
- Operacja `pop_back()` zwraca zawartość ostatniej zapełnionej komórki pamięci wektora oraz powoduje jej usunięcie, tzn. kolejno:
 - zmniejszenie wartości n o 1,
 - realokację (zmniejszenie) bloku pamięci w przypadku, gdy $n << a$ (zazwyczaj, gdy: $n \simeq a/4$).
- Bardzo dobrą wydajność czasową uzyskujemy, gdy przy realokacjach powiększających podwajamy rozmiar bloku `ptr` ($a = 2*a$), a przy zmniejszających – zmniejszać go dwukrotnie ($a = a/2$).

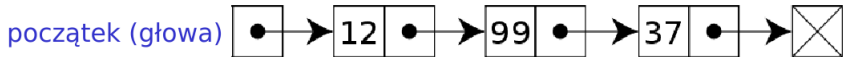
Implementacja wektora oraz funkcji do jego obsługi w języku C: program `w10p05.c` – do pobrania ze strony przedmiotu.

Lista jednokierunkowa i dwukierunkowa

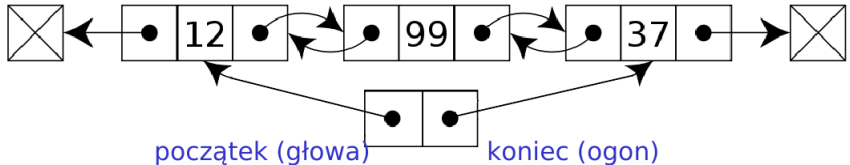
Listą nazywamy strukturę danych, w której elementy są uporządkowane liniowo, a każdy element listy obejmuje:

- Wartość w nim przechowywaną.
- Wskaźnik (wskazanie) na kolejny element listy.
- Wskaźnik (wskazanie) na poprzedni element listy – tylko w przypadku list dwukierunkowych.

Ilustracja listy jednokierunkowej:



Ilustracja listy dwukierunkowej:



Uwagi o listach

- Zmienna „opisująca” listę w programie to tak naprawdę wskaźnik na początkowy element listy. Opcjonalnie, można dodatkowo przechowywać wskaźnik na końcowy element listy i/lub liczbę elementów listy.
- W czasie stałym można zrealizować operację wstawienia bądź usunięcia elementu na początku listy (jak również na końcu listy, o ile dysponujemy wskaźnikiem do niego).
- Lista nie oferuje dostępu swobodnego, tzn. w celu wypisania wartości jej j -tego elementu należy iterować (przemieszczać się) j razy po kolejnych elementach listy, rozpoczynając od jej początku.
 - W przypadku list dwukierunkowych możliwe jest również iterowanie odpowiednią liczbę razy od końca listy.
 - Dodanie nowego elementu listy bezpośrednio po elemencie bieżącym (tzn. takim, do którego dysponujemy wskaźnikiem) można zrealizować w czasie stałym.
 - Usunięcie bieżącego elementu listy dwukierunkowej można zrealizować w czasie stałym. W liście jednokierunkowej jest to również możliwe, o ile dysponujemy wskaźnikiem do elementu poprzedzającego.
- Użycie listy najczęściej wiąże się z alokacją każdego elementu listy w odrębnym, oddzielnie alokowanym bloku pamięci.

Przykład implementacji listy jednokierunkowej – definicja i inicjalizacja

```
typedef struct list {
    int value;
    struct list *next;
} list_t;

// Inicjalizuje listę o głowie "head".
void init( list_t *head ) {
    head->next = NULL;
}
```

- Element typu `list_t` zawiera “pole danych”, które nazywamy `value` oraz wskaźnik `next` na tego typu strukturę.
- Jeśli dysponujemy wskaźnikiem `ptr` na strukturę typu `list_t`, która jest elementem listy, to mamy dwie możliwości:
 - Jeśli `ptr == NULL`, to `ptr` wskazuje na ostatni element listy.
 - Jeśli `ptr != NULL`, to `ptr` nie jest ostatnim elementem listy. Wówczas, adres kolejnego elementu listy to `ptr->next`.

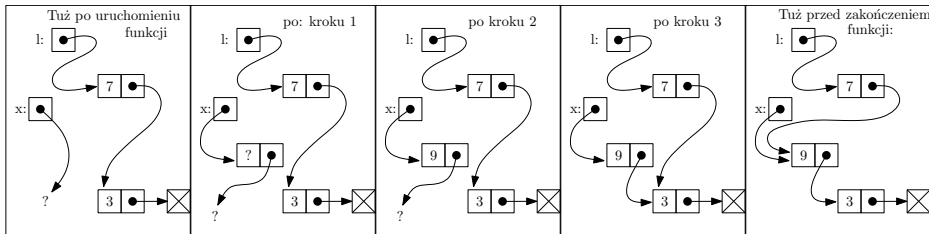
Tą własność musimy zagwarantować odpowiednią implementacją funkcji, które wstawiają i usuwają elementy listy.

- W prezentowanej implementacji, pierwszy element listy ma nieokreśloną wartość pola `value`. Implementacja jest wykonana w ten sposób, aby wstawianie i usuwanie elementów nigdy nie dokonało zmiany pierwszego jej elementu. Takie podejście nie jest konieczne, lecz ułatwia przekazywanie parametrów do funkcji podanych na kolejnych slajdach.

Przykład implementacji listy jednokierunkowej – wstawianie elementu

```
void add_after( list_t *l, int value ) {  
    list_t *x = (list_t *) malloc( sizeof(list_t) ); // Krok 1.  
  
    x->value = value;      // Krok 2.  
    x->next = l->next;    // Krok 3.  
    l->next = x;           // Krok 4.  
}
```

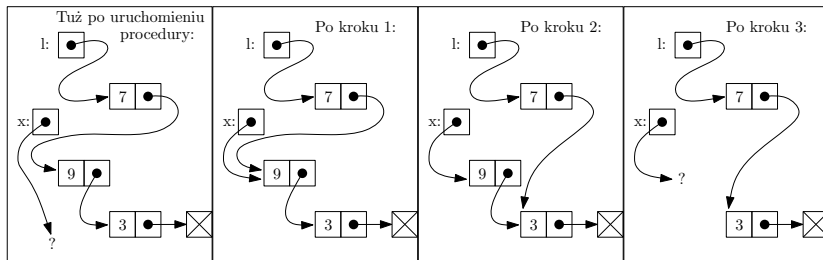
- Załóżmy, że wywołujemy `add_after( l, 9 )`, gdzie `l` wskazuje na pewien element listy.
- Funkcja “wstawi” nowy element listy tak, by był następnikiem elementu wskazywanego przez `l`.
- Nowy element będzie miał wartość `value` równą 9.
- **Uwaga:** funkcję warto uzupełnić o sprawdzenie czy wywołanie `malloc` powiodło się.
- Kolejne etapy działania procedury można przedstawić następująco:



Przykład implementacji listy jednokierunkowej – usuwanie elementu

```
void remove_after( list_t *l ) {  
    if ( l->next != NULL ) {  
        void* x = l->next;           // Krok 1.  
        l->next = l->next->next;      // Krok 2.  
        free( x );                    // Krok 3.  
    }  
}
```

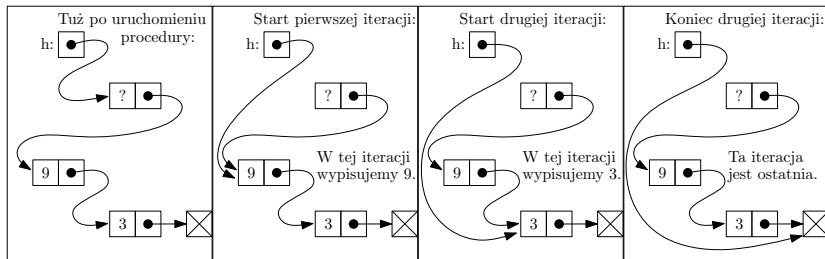
- Załóżmy, że wywołujemy `remove_after( l )`, gdzie `l` wskazuje na pewien element listy.
- Funkcja “usunie” z listy element wskazywany przez `l->next`, czyli “następnik” `l`, o ile takowy istnieje. Należy pamiętać o zwolnieniu pamięci.
- Zauważ, że nie możemy od razu wywołać `free(l->next)`, gdyż wówczas nie moglibyśmy “powiązać” elementów listy.
- Kolejne etapy działania procedury można przedstawić następująco:



Przykład implementacji listy jednokierunkowej – przeglądanie listy

```
void print( list_t *h ) {  
    h = h->next;  
    while ( h != NULL ) {  
        printf ( "%d ", h->value);  
        h = h->next;  
    }  
}
```

- Załóżmy, że wywołujemy `print( h )`, gdzie `h` wskazuje na “głową” listy.
- Zgodnie z naszą implementacją, głowa listy zawiera nieokreśloną wartość pola `value`.
- Zauważ, że funkcja dysponuje kopią adresu głowy listy, więc modyfikując `h` nie zmieniamy wartości adresu przechowanej w zmiennej przekazanej jako argument.
- Kolejne etapy działania procedury można przedstawić następująco:



Stos

Stosem nazywamy strukturę danych, udostępniającą dwie operacje:

- Operacja **push(x)** powoduje wstawienie wartości x na wierzchołek stosu.
- Operacja **pop()** zwraca wartość znajdującą się na wierzchołku stosu oraz usuwa tę wartość ze stosu.

Uwagi:

- Wartości zawsze zdejmowane są ze stosu w kolejności odwrotnej do kolejności ich umieszczania na stosie (LIFO – last in, first out).
- Wektor może zostać użyty do implementacji stosu (**push_back** realizuje operację **push**, **pop_back** realizuje **pop**).
 - Zaleta implementacji stosu na wektorze: jeden ciągły blok pamięci.
 - Wada: operacje **push_back**/**pop_back** średnio rzecz biorąc działają szybko, ale czasem mogą trwać bardzo długo (gdy wykonuje się realokacja pamięci).
- Lista jednokierunkowa może zostać użyta do implementacji stosu:
 - Każdy element stosu jest umieszczany w odrębnym, alokowanym dla niego bloku pamięci. Wada: wielokrotne alokacje/realokacje małych bloków pamięci.
 - Czas wykonania każdej operacji **push**/**pop** jest wówczas stały.

Implementacja stosu z wykorzystaniem listy jednokierunkowej: program w10p07.c – do pobrania ze strony przedmiotu.

Kolejka

Kolejką nazywamy strukturę danych, udostępniającą dwie operacje:

- Operacja **add(x)** powoduje wstawienie wartości x na koniec kolejki.
- Operacja **remove()** zwraca wartość znajdującą się na początku kolejki oraz usuwa tę wartość z kolejki.

Uwagi:

- Wartości zawsze usuwane z kolejki w kolejności ich umieszczania w kolejce (FIFO – first in, first out).
- Lista jednokierunkowa może zostać użyta do implementacji kolejki:
 - Każdy element kolejki jest umieszczany w odrębnym, alokowanym dla niego bloku pamięci. Wada: wielokrotne alokacje/realokacje małych bloków pamięci.
 - Operacja **add** dodaje element na koniec listy – powinniśmy dysponować adresem ostatniego elementu, aby operację tą wykonywać szybko.
 - Operacja **remove** usuwa pierwszy element listy.
 - Czas wykonania każdej operacji **add/remove** jest wówczas stały.
- Wektor nie nadaje się do implementacji kolejki.
- Implementacja kolejki przy użyciu tablicy jest możliwa. Stosuje się wówczas tzw. mechanizm cykliczny.

Podstawy Programowania

Wykład nr 11: Wybrane techniki implementacji algorytmów

dr hab. inż. Dariusz Dereniowski

Katedra Algorytmów i Modelowania Systemów
Wydział ETI, Politechnika Gdańska

Spamiętywanie

Spamiętywanie – technika optymalizacji rekurencji, pozwalająca na zapamiętaniu wyniku (wartości zwracanej) wywołania funkcji z danym zestawem parametrów i przywołania tej wartości przy późniejszym wywołaniu z tym samym zestawem parametrów:

- spamiętywanie może być wykorzystane tylko w przypadku funkcji obliczeniowych, w których istotna jest jedynie wartość zwracana, a nie pojawiają się efekty uboczne (takie jak: modyfikacja obszarów pamięci innych procedur, operacje wejścia/wyjścia),
- spamiętywanie może znacząco obniżyć złożoność czasową (a tym samym czas działania programu) i pamięciową (a tym samym ilości pamięci wykorzystywanej przez program) algorytmu,
- spamiętywanie może się wiązać z koniecznością implementacji dodatkowych struktur danych w celu zapamiętania wyników wcześniejszych wywołań funkcji (pewien narzut czasowy i pamięciowy).

Spamiętywanie – przykład

```
#define MAX 55
```

```
/* Funkcja obliczająca wartość n-tej liczby Fibonacciego (wersja rekurencyjna) */
```

```
long long fib_rek( int n ) {  
    if ( n <= 2 )  
        return 1;  
    else  
        return fib_rek( n-2 ) + fib_rek( n-1 );  
}
```

```
/* Funkcja obliczająca wartość n-tej liczby Fibonacciego (spamiętywanie).  
   Funkcja korzysta z globalnej tablicy t, w której:  
   t[i] == 0 oznacza, że i-ta l. Fibonacciego nie została jeszcze obliczona  
   t[i] != 0 oznacza, że i-ta l. Fibonacciego została wcześniej  
   obliczona i użycie rekurencji nie jest potrzebne. */
```

```
long t[MAX];  
long long fib_dyn( int n ) {  
    if ( n <= 2 )  
        return t[1]=t[2]=1;  
    else {  
        if ( t[n-2] == 0 )  
            fib_dyn( n-2 );  
        if ( t[n-1] == 0 )  
            fib_dyn( n-1 );  
        t[n] = t[n-2] + t[n-1];  
        return t[n];  
    }  
}
```

Spamiętywanie a rekurencja – porównanie wydajności

```
#include <stdio.h>
#include <time.h>
#include <string.h>
#define MAX 55

long long fib_rek( int n );

long t[MAX];
long long fib_dyn( int n );

int main() {
    long long f;
    clock_t start, fib_rek_czas, fib_dyn_czas;

    printf( "%2s | %15s | %10s | %10s (czas1 = czas wersji rek., czas2 = czas wersji dyn.)\n", "n", "n", "wart.", "czas1", "czas2" );

    for ( int n=35; n < MAX; n+=2 ) {
        memset( (void *)t, 0, sizeof t ); // Zerujemy tablicę t do nowego testu.

        start = clock(); // Mierzmy czas działania fib_rek(n).
        f = fib_rek( n ); // Czas w sekundach będzie wynosił:
        fib_rek_czas = clock() - start; // ((float)fib_rek_czas)/CLOCKS_PER_SEC

        start = clock();
        fib_dyn( n );
        fib_dyn_czas = clock() - start;

        printf( "%2d | %15lld | %10.2f | %10.2f\n", n, f, ((float)fib_rek_czas)/CLOCKS_PER_SEC, ((float)fib_dyn_czas)/CLOCKS_PER_SEC );
    }
    return 0;
}
```

n	wart.	czas1	czas2
35	9227465	0.05	0.00
37	24157817	0.15	0.00
39	63245986	0.38	0.00
41	165580141	0.97	0.00
43	433494437	2.55	0.00
45	1134903170	6.61	0.00
47	2971215073	17.28	0.00
49	7778742049	45.31	0.00
51	20365011074	121.54	0.00
53	53316291173	313.72	0.00

Programowanie dynamiczne

Programowanie dynamiczne – technika polegająca na połączeniu spamiętywania i derekursywacji, która może być zastosowana dla problemów, w których rozwiązanie uzyskuje się poprzez rozszerzenie rozwiązania dla podproblemu (tzw. własność optymalnej podstruktury). W szczególności:

- punktem wyjścia jest zazwyczaj rozwiązanie rekurencyjne dla danego problemu,
- rekurencja charakteryzuje się m.in. tym, że rozwiązanie problemu jest sprowadzane do rozwiązania mniejszych podproblemów, z których to rozwiązań “budowane” jest rozwiązanie oryginalnego problemu,
- często się zdarza, że w toku obliczeń funkcja rekurencyjna jest wywoływana wielokrotnie dla tego samego podproblemu,
- w programowaniu dynamicznym rozwiązanie danego podproblemu wyznaczamy tylko raz, zapamiętując je w tablicy,

```
#define MAX 100
long t[MAX];
long long fib_dyn( int n ) {
    t[1] = t[2] = 1;
    for ( int i=3; i <= n; i++ )
        t[i] = t[i-2] + t[i-1];
    return t[n];
}
```

Uwaga: podany program zakłada, że funkcja nie zostanie wykonana z argumentem większym niż 99!

Ćwiczenie: napisz funkcję, która nie wykorzystuje ani tablic ani rekurencji do obliczenia zadanej liczby Fibonacciego.

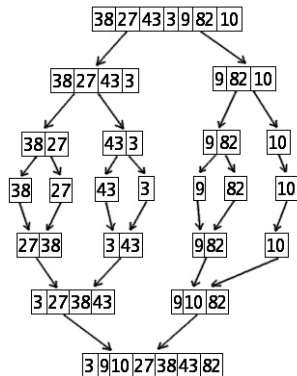
Technika „dziel i zwyciężaj”

- Algorytm typu „dziel i zwyciężaj” rekurencyjnie rozkłada problem do rozwiązania na przynajmniej dwa mniejsze podproblemy, aż do momentu, gdy podproblem nadaje się do rozwiązania wprost.
- Na każdym poziomie rekurencji, rozwiązania podproblemów są „składane”, by uzyskać rozwiązanie większego problemu.

Przykład “Dziel i zwyciężaj” – sortowanie

Przykład: sortowanie tablicy liczb przez scalanie (**MergeSort**)

- Podziel tablicę na dwie (prawie) równe części.
- Wywołaj rekurencyjnie procedurę **MergeSort** dla każdej z nich oddzielnie, chyba że pozostał już tylko jeden element.
- Scal dwa posortowane podciągi.



Przykład “Dziel i zwyciężaj” – wyszukiwanie w posortowanej tablicy

- Jeśli tablica jest posortowana, to problem wyszukiwania zadanego elementu można znacznie przyspieszyć.
- W metodzie **wyszukiwania binarnego**, szukany element jest porównywany ze środkowym elementem w tablicy.
- Jeśli tak się zdarzy, że ich wartości są równe, to szukany element został znaleziony.
- W przeciwnym wypadku, kontynuujemy poszukiwanie (rekurencyjnie) w odpowiedniej “połówce” tablicy.

```
typedef int element_t;

// F-cja zwraca indeks i tż t[i]==szukany, lub -1 jeśli takie i nie istnieje.
int BinarySearch( element_t szukany, element_t *t, int lewy, int prawy ) {
    if ( lewy > prawy )
        return -1; // Szukanego elementu nie ma w tablicy.
    else {
        int srodek = (lewy + prawy)/2;
        if ( t[srodek] == szukany )
            return srodek;
        else if ( t[srodek] > szukany )
            return BinarySearch( szukany, t, lewy, srodek-1 );
        else
            return BinarySearch( szukany, t, srodek+1, prawy );
    }
}
```

Przykład “Dziel i zwyciężaj” – QuickSort (“szybkie sortowanie”)

Zasada sortowania tablicy t rozmiaru n metodą QuickSort:

- wybieramy dowolny element x w tablicy t ,
- Tworzymy podział tablicy na dwie części: część składająca się z elementów mniejszych od x oraz z pozostałych elementów.
- Podział tworzony jest tak, że tablica t zostaje “przearanżowana” w taki sposób, że wszystkie elementy mniejsze od x są położone na lewo od x (x może zmienić położenie w stosunku do położenia pierwotnego w t).
- Załóżmy, że po “rearanżacji” x znajduje się pod indeksem j w t , tzn. $t[j]=x$.
- Zauważmy, że zadanie sortowania wyjściowego ciągu n liczb $t[0], \dots, t[n-1]$ zostało “zredukowane” do sortowania dwóch podciągów $t[0], \dots, t[j-1]$ oraz $t[j+1], \dots, t[n-1]$, każdy o długości mniejszej niż n .
- Rekurencyjne posortowanie dwóch powyższych podciągów prowadzi do tego, że tablica t będzie posortowana.

Przykładowa implementacja QuickSort

```
void zamien( int *t, int i1, int i2 ) { // Funkcja zamienia miejscami
    int a = t[i1]; // elementy o indeksach i1, i2
    t[i1] = t[i2]; // w tablicy t.
    t[i2] = a;
}

/* Funkcja dokonuje podziału tablicy t, w którym element t[lewy] zostanie umieszczony pod
indeksem granica, gdzie lewy <= granica <= prawy. Ponadto, t[i] <= t[granica] dla
wszystkich indeksów lewy <= i < granica, oraz t[j] >= t[granica] dla wszystkich
indeksów granica < j <= prawy.*/
int podzial( int *t, int lewy, int prawy ) {
    int x = t[lewy], granica = lewy;
    zamien( t, lewy, prawy );
    for ( int i=lewy; i < prawy; i++ )
        if ( t[i] <= x )
            zamien( t, i, granica++ );
    zamien( t, granica, prawy );
    return granica;
}

/* Funkcja sortująca 'fragment' tablicy t pod indeksami lewy, ..., prawy.
Aby posortować tablicę A należy wywołać:
QuickSort( A, 0, (sizeof A)/(sizeof A[0])-1 ); */
void QuickSort( int *t, int lewy, int prawy ) {
    if ( lewy < prawy ) {
        int granica = podzial( t, lewy, prawy );
        QuickSort( t, lewy, granica-1 );
        QuickSort( t, granica+1, prawy );
    }
}
```

Wskaźniki do funkcji

Deklaracja wskaźnika do funkcji o argumentcie typu `int` oraz zwracającej typ `double` (analogicznie dla innych typów argumentów i typów zwracanych wartości):

```
double (*wsk_fun)(int x)
```

Jeśli w programie mamy funkcję o nazwie `g`, której prototyp zgadza się z powyższym (czyli `g` przyjmuje argument typu `int` oraz zwraca wartość typu `double`), to możemy dokonać przypisania:

```
wsk_fun = g;
```

Wówczas, równoważne formy wywołania funkcji `g`:

```
g( 5 );  
(*wsk_fun)( 5 );  
wsk_fun( 5 );
```

Wskaźniki do funkcji – przykład wywołania bibliotecznego qsort

```
#include <stdio.h>
#include <stdlib.h>

int cmp (const void * a, const void *b) {
    return *(int*)a - *(int*)b;

    /* Porównaj :
    int _va = *(int*)a;
    int _vb = *(int*)b;

    if (_va > _vb) return 1;
    if (_va < _vb) return -1;
    return 0;
    */
}

int main() {
    int t[] = { 6, 34, 21, 43, 34, 1, 0, 23, 12, 22 };

    qsort ((void*) t, (sizeof t)/(sizeof t[0]), sizeof(int), cmp);

    for (int i=0; i<(sizeof t)/(sizeof t[0]); i++)
        printf ("%d ", t[i]);
    putchar( '\n' );
    return 0;
}
```

Diagnostyka programów – warunkowe dyrektywy

```
#ifdef IDENTYFIKATOR
```

```
...
```

```
#endif
```

Fragment ... podlega kompilacji tylko wówczas jeśli identyfikator **IDENTYFIKATOR** jest zdefiniowany.

```
#ifndef IDENTYFIKATOR
```

```
...
```

```
#endif
```

Fragment ... podlega kompilacji tylko wówczas jeśli identyfikator **IDENTYFIKATOR** nie jest zdefiniowany.

- Powyższy mechanizm można użyć do umieszczania kodu, który chcemy wykonywać tylko w fazie testowania programu.
- Aby to osiągnąć, definiujemy identyfikator, na przykład **#define TEST**.
- Następnie, wewnątrz dyrektywy **#ifdef TEST ... #endif** umieszczamy dodatkowy kod używany w fazie testowania programu.
- Gdy chcemy dokonać kompilacji docelowego kodu, usuwamy tylko linię zawierającą definicję identyfikatora **TEST**.

Kompilacja warunkowa – przykład

```
#include <stdio.h>
#include <stdlib.h>

int cmp (const void * a, const void *b) {
    #ifndef TEST
        int a1 = *(int*)a, b1 = *(int*)b;
        printf( " %d %c %d\n", a1,
            (a1==b1 ? '=' : (a1>b1 ? '>' : '<')) , b1 );
    #endif
    return  *(int*)a - *(int*)b;
}

int main() {
    int t[] = { 6, 34, 21, 43, 34, 1, 0, 23, 12, 22 };

    qsort( (void*)t, (sizeof t)/(sizeof t[0]), sizeof(int), cmp );

    for (int i=0; i<(sizeof t)/(sizeof t[0]); i++)
        printf ("%d ", t[i]);
    putchar( '\n' );
    return 0;
}
```

```
6 < 34
43 > 34
21 < 34
6 < 21
34 > 21
34 = 34
1 > 0
12 < 22
23 > 12
23 > 22
0 < 12
1 < 12
6 > 0
6 > 1
6 < 12
21 > 12
21 < 22
34 > 22
34 > 23
0 1 6 12 21 22 23 34 34 43
```

Kompilacja warunkowa – przykład

```
#include <stdio.h>
#include <stdlib.h>

// Przypisanie konkretnej wartości identyfikatorowi TEST nie jest tutaj konieczne.
#define TEST

int cmp (const void * a, const void *b) {
    #ifndef TEST
        int a1 = *(int*)a, b1 = *(int*)b;
        printf( " %d %c %d\n", a1, (a1==b1 ? '=' : (a1>b1 ? '>' : '<') ), b1 );
    #endif
    return  *(int*)a - *(int*)b;
}

int main() {
    int t[] = { 6, 34, 21, 43, 34, 1, 0, 23, 12, 22 };

    qsort( (void*)t, (sizeof t)/(sizeof t[0]), sizeof(int), cmp);

    for (int i=0; i<(sizeof t)/(sizeof t[0]); i++)
        printf( "%d ", t[i] );
    putchar( '\n' );
    return 0;
}
```

0 1 6 12 21 22 23 34 34 43

Uwaga: W tym przypadku instrukcje “wewnątrz” dyrektywy są usuwane z kodu przed kompilacją. Nie jest to więc równoważne użyciu instrukcji warunkowej: instrukcja warunkowa jest zawsze tłumaczona na kod wykonywalny podczas kompilacji, podczas gdy powyższa dyrektywa tylko “steruje” selekcją kodu, który będzie poddany kompilacji.